

조건을 설계하라

불 대수, 논리게이트와 Python으로 만드는 스마트 발명품

김영준

서울과학고등학교

developer25@sshs.hs.kr

강의 목표

- 생활 속 문제를 사용자, 입력 조건, 출력 행동으로 나누어 분석할 수 있다.
- 장치의 작동 조건을 0과 1, 진리표, 논리식으로 표현할 수 있다.
- 논리게이트를 이용하여 스마트 장치의 판단 회로를 설계할 수 있다.
- 불 대수의 동치 관계를 이용하여 같은 기능의 서로 다른 설계를 비교하고, 부품 수와 부품 종류의 관점에서 더 경제적인 설계를 고를 수 있다.
- Python에서 `and`, `or`, `not`을 사용하여 판단 함수를 구현할 수 있다.
- tkinter로 가상 센서와 출력 장치를 갖춘 디지털 프로토타입을 만들 수 있다.
- 전체 입력 조합을 시험하여 오작동을 찾아내고, 작동 조건과 사용자 인터페이스를 개선할 수 있다.

주제어: 불 대수, 불 함수, 진리표, 논리식, 논리게이트, 함수적 완전성, NAND, 스마트 발명품, 판단 규칙, 가상 센서, Python, tkinter, 디지털 프로토타입, 제품 테스트

수업 운영 계획 (50분 × 3차시 × 2일)

일자	차시	주제	핵심 활동	산출물
1일차 개인 활동	1	발명품의 작동 조건	입력과 출력 정의, 0/1 모델링, NOT/AND/OR	조건식
	2	진리표와 논리게이트	진리표와 논리식, 회로, 반가산기와 전가산기	진리표와 회로
	3	Python 개인 실습	공통 파일 실습과 테스트, 우산 GUI, 개인 아이디어 카드	개인 아이디어 카드
2일차 팀 활동	1	아이디어 공유와 팀 구성	개인 발표와 둘러보기, 2~4명 팀 구성, 최종 진리표	팀 발명 설계서
	2	tkinter 팀 프로토타입	TEST_CASES, <code>decide()</code> , GUI 순서로 구현	tkinter 프로토타입
	3	테스트와 발명적 최적화	모든 입력 조합 시험, UI 개선, 교차 테스트, 발표	테스트 및 개선 기록

1일차의 목표는 “내가 논리 시스템을 만들고 검증할 수 있다”이다. 각자 불 대수와 논리게이트를 배우고 공통 Python 파일을 시험한 뒤, 자신의 관심과 발명 아이디어를 카드에 정리한다. 2일차의 목표는 “우리의 아이디어를 논리와 코드로 검증하여 프로토타입으로 만든다”이다. 개인 아이디어를 공유하고 팀을 만든 뒤, 팀이 합의한 논리를 Python과 tkinter로 구현한다.

2일차 프로젝트는 두 트랙 중 하나를 고른다. **A 트랙**은 생활 속 문제를 해결하는 스마트 발명품을 만들고, **B 트랙**은 1일차에 유도한 전가산기를 이진 계산 장치로 만든다. 1일차 내용은 두 트랙이 공통이며, 트랙은 2일차 아이디어 공유와 팀 구성 뒤에 정한다.

실습 환경

항목	내용
실행 환경	로컬에 설치된 Python 3
그래픽 라이브러리	Python 표준 라이브러리 tkinter, tkinter.ttk
설치할 패키지	없음. 외부 패키지를 설치하지 않는다
편집기	제한하지 않음. IDLE, VS Code 등 익숙한 편집기를 사용한다
사전 점검	<code>python --version</code> 과 <code>python -m tkinter</code> 실행
실습 파일	code/ 디렉터리의 Python 파일 (README.md에 역할 정리)

`python` 명령이 인식되지 않는 Windows 환경에서는 `py` 명령을 사용한다. `python -m tkinter`를 실행하여 작은 창이 열리면 실습 준비가 끝난 것이다.

목차

1 불 대수: 발명품의 작동 조건 설계	4
1.1 발명품의 입력과 출력	4
1.2 참·거짓과 0·1	4
1.3 NOT, AND, OR	5
1.4 진리표는 제품 테스트표다	6
1.5 불 함수는 장치의 판단 규칙이다	7
1.6 진리표에서 논리식 만들기	7
1.7 같은 기능, 다른 설계	8
1.8 두 상태가 다를 때만: XOR	9
1.9 발명 설계 미션	10
2 논리게이트: 판단 회로에서 이진 덧셈까지	12
2.1 불 대수에서 전기 회로로	12
2.2 NOT, AND, OR 게이트	12
2.3 논리식을 회로로 바꾸기	13
2.4 스마트 과학실 안전관리 시스템	13
2.5 같은 기능을 더 경제적으로	15
2.6 NAND와 부품 표준화 [심화]	15
2.7 논리게이트로 이진수를 더한다	17
2.8 2일차 프로젝트 미리 보기	19
3 Python 논리 실험실	22
3.1 실습 환경 확인	22
3.2 불 대수와 Python의 번역	22
3.3 판단 함수 만들기	24
3.4 Python으로 진리표 생성하기	24
3.5 테스트 코드는 제품 시험표다	25
3.6 고장 난 판단 함수 수리하기	27
3.7 GUI 맛보기와 개인 아이디어 준비	28
4 tkinter 프로토타입 제작 Lab	29
4.1 개인 아이디어 공유와 팀 구성	29
4.2 왜 tkinter로 프로토타입을 만드는가	29
4.3 입력-판단-출력 구조	30
4.4 tkinter의 최소 개념	30
4.5 코드의 두 영역	31
4.6 Lab 1. 공통 실습: 우산 챙김 알림 장치	31
4.7 Lab 2 (A 트랙). 팀별 스마트 발명품	33
4.8 Lab 2 (B 트랙). 이진 계산 장치	36
4.9 완성 예시: 스마트 과학실 안전관리 시스템	39
4.10 전체 입력 테스트	40
4.11 사용자 인터페이스 개선	41

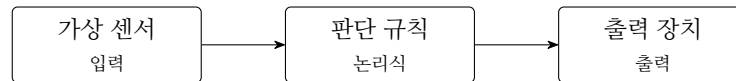
4.12	모뎀 간 교차 테스트	42
4.13	1분 발표	43
A	수학 활동 정답 및 해설	44
A.1	예제 1.1 스마트 복도 조명의 세 가지 설계	44
A.2	예제 1.2 우산 챙김 알림 장치의 진리표	44
A.3	예제 1.3 스마트 사물함 출입 제어	45
A.4	예제 1.4 같은 기능, 다른 설계	45
A.5	도전 문제 1.2 XOR를 NOT과 AND로	46
A.6	연습문제 1.3 진리표에서 논리식 구성	46
A.7	예제 2.1 스마트 과학실 안전관리 시스템	47
A.8	예제 2.2 낭비를 찾아내기	47
A.9	예제 2.3 NAND만으로 AND와 OR 만들기	47
A.10	예제 2.4 합과 올림을 논리식으로	48
A.11	연습문제 2.3 전가산기의 진리표	48
A.12	도전 문제 2.2 우리 팀 회로를 NAND만으로	48
B	Python Lab 정답 및 예상 출력	50
B.1	실습 문제 3.2 세 가지 연산	50
B.2	실습 문제 3.3 값을 바꾸어 실험하기	50
B.3	실습 문제 3.4 진리표 자동 생성과 테스트	50
B.4	실습 문제 3.6 오작동 수리 절차	50
B.5	도전 문제 3.7 진리표에서 논리식 되찾기	51
B.6	Lab 정답 파일	52
B.7	B 트랙 전가산기의 혼한 오작동	53
B.8	도전 문제 4.4 두 자리 이진 덧셈기	53
C	tkinter 교사용 운영 안내	54
C.1	수업 전 준비	54
C.2	자주 발생하는 오류와 대응	55
C.3	진행 속도가 다른 모듈 지원	55
C.4	강의안의 코드와 실습 파일 동기화	56
D	수학 심화	57
D.1	XOR는 범용 게이트인가	57
D.2	함수적 완전성과 범용 게이트	57
D.3	NAND만으로 만든 반가산기	58
D.4	전가산기 설계	58
E	평가 기준	60
E.1	채점 시 유의점	60

1. 불 대수: 발명품의 작동 조건 설계

오늘의 목표. 복도의 조명은 하루 종일 켜져 있고, 비 오는 날 우산을 두고 나가는 사람은 늘 있다. 두 문제 모두 “장치가 언제 작동해야 하는가”를 정확히 정하지 못해서 생긴다. 어떤 조건에서 장치가 작동해야 하는지 누구도 다르게 해석할 수 없도록 정확하게 적는 방법이 필요하다. 그 방법이 불 대수다. 1일차 앞부분에서는 이 언어를 배우고 같은 언어로 회로를 설계하며, 마지막에는 그 규칙을 Python으로 옮겨 검증한다.

1.1. 발명품의 입력과 출력

스마트 발명품은 대개 세 부분으로 이루어진다.



입력은 장치가 알아낼 수 있는 사실이다. 사람이 있는지, 어두운지, 비가 오는지처럼 센서가 감지할 수 있는 것이어야 한다. 출력은 장치가 실제로 하는 행동이다. 조명을 켜거나, 경보를 울리거나, 환풍기를 돌리는 것이 출력이다. 판단 규칙은 어떤 입력 상태에서 출력을 작동시킬지 정하는 규칙이다.

발명에서 가장 자주 틀리는 부분은 판단 규칙이다. “어두우면 켜진다”처럼 대충 적어 두면, 아무도 없는 새벽에도 조명이 켜진다. 그래서 판단 규칙은 말이 아니라 식으로 적어야 한다.

1.2. 참·거짓과 0·1

입력은 언제나 “그렇다”와 “아니다” 둘 중 하나로 답할 수 있는 문장이어야 한다. 이런 문장을 명제(*proposition*)라 하고, 보통 P , Q , R 과 같은 기호로 나타낸다. 이 강의에서는 다음 명제를 계속 사용한다.

기호	뜻	기호	뜻
P	사람이 있다	U	우산을 가지고 있다
D	주변이 어둡다	G	가스가 감지되었다
R	비가 온다	H	온도가 지나치게 높다
O	외출할 예정이다	C	학생 카드가 인식되었다

명제의 진리값은 참(true) 또는 거짓(false)이며 수학에서는 T 와 F 로 쓴다. 그런데 센서와 회로, 그리고 Python으로 이어지려면 다음 대응을 쓰는 편이 훨씬 편하다.

값	진리값	센서의 상태	출력 장치의 상태	전기 신호
0	거짓 F	감지되지 않음	꺼짐	신호 없음
1	참 T	감지됨	켜짐	신호 있음

앞으로 진리표는 0과 1로 쓰고, 행의 순서는 항상 작은 수부터 00, 01, 10, 11 순서로 적는다. 입력이 세 개이면 000, 001, 010, 011, 100, 101, 110, 111 순서다. 이렇게 정해 두면 빠뜨린 경우가 있는지 바로 알 수 있다.

참고. 실제 센서가 처음부터 0과 1만 내놓는 것은 아니다. 온도 센서는 28.4°C 같은 값을 재고, 조도 센서는 밝기를 숫자로 알려 준다. 그 값을 0 또는 1로 바꾸는 것은 설계자의 몫이다. “ 30°C 이상이면 $H = 1$ ”처럼 기준을 정하는 순간 측정값이 하나의 명제가 되고, 비로소 불 대수로 다룰 수 있다. 기준을 어디에 두느냐에 따라 장치의 작동이 달라지므로, 기준을 정하는 일도 설계의 일부다.

발명 노트. 19세기 영국의 수학자 조지 불(George Boole)은 논리도 대수처럼 다룰 수 있다고 생각했다. 그는 참과 거짓을 값으로, “그리고”와 “또는”을 연산으로 보면 논리 법칙을 식으로 쓸 수 있음을 보였다. 이 아이디어를 따서 오늘날 불 대수(*Boolean algebra*)라고 부른다. 당시에는 순수한 논리 연구였지만, 약 70년 뒤 이 대수는 전기 회로의 설계 언어가 되었고, 지금 우리가 만드는 스마트 발명품의 판단 규칙도 같은 언어로 쓰인다.

1.3. NOT, AND, OR

작동 조건을 만드는 데 필요한 연산은 세 가지뿐이다.

정의 1 (Not). 명제 P 의 부정(*not*)을 $\neg P$ 로 쓴다.¹ P 가 참이면 $\neg P$ 는 거짓이고, P 가 거짓이면 $\neg P$ 는 참이다.

P	$\neg P$
0	1
1	0

장치 관점. NOT은 센서 상태를 반대로 해석한다. “우산이 없을 때”라는 조건은 우산 센서 U 를 그대로 쓰지 않고 $\neg U$ 로 써야 한다.

정의 2 (And). 두 명제가 모두 참일 때만 참이 되는 연산을 논리곱(*and*)이라 하고 $P \wedge Q$ 로 쓴다.

P	Q	$P \wedge Q$
0	0	0
0	1	0
1	0	0
1	1	1

장치 관점. AND는 “모든 조건이 동시에 만족될 때만 작동”을 뜻한다. 꼭 필요할 때만 작동시켜 전기나 재료를 아끼려면 AND를 쓴다.

정의 3 (Or). 두 명제 중 적어도 하나가 참이면 참이 되는 연산을 논리합(*or*)이라 하고 $P \vee Q$ 로 쓴다.

P	Q	$P \vee Q$
0	0	0
0	1	1
1	0	1
1	1	1

장치 관점. OR는 “여러 위험 조건 중 하나라도 감지되면 작동”을 뜻한다. 안전 장치는 대개 OR로 시작한다. 둘 다 참인 경우에도 참이므로, 일상의 “또는”과 달리 “둘 중 하나만”을 뜻하지 않는다. 이런 의미의 논리합을 포함적 논리합(*inclusive or*)이라고 한다.

예제 1.1 스마트 복도 조명의 세 가지 설계

학교 복도 조명이 낮에도 켜져 있어 전기가 낭비된다. 입력은 P (사람이 있다)와 D (주변이 어둡다)이고, 출력은 L (조명이 켜진다)이다. 다음 세 설계를 비교하자.

설계 1: $L = P$ 설계 2: $L = D$ 설계 3: $L = P \wedge D$

¹우리나라 교육과정에서는 부정을 $\sim P$ 로 나타내기도 한다.

- (1) 사람이 없는데도 조명이 켜지는 설계는 무엇인가?
- (2) 밝은 낮에도 조명이 켜지는 설계는 무엇인가?
- (3) 전기 낭비를 가장 잘 줄이는 설계는 무엇인가?
- (4) 설계 3에 “관리자가 허가했다”는 조건까지 AND로 더 붙이면 어떤 문제가 생길 수 있는가?

(4)의 답은 발명에서 매우 중요하다. 조건을 지나치게 엄격하게 만들면 정작 필요한 순간에 장치가 작동하지 않는다. 조건을 느슨하게 하면 쓸데없이 작동한다. 좋은 설계는 이 둘 사이를 맞추는 일이다.

1.4. 진리표는 제품 테스트표다

입력이 n 개이면 가능한 입력 조합은 2^n 가지다. 이 모든 경우에 대해 출력을 적은 표가 진리표(*truth table*)다.

진리표는 단순한 계산표가 아니다. 진리표는 발명품이 모든 입력 상황에서 의도대로 작동하는지 확인하는 **제품 테스트표**다.

우리 장치처럼 입력이 2~3개인 경우에는 가능한 모든 입력 조합을 빠짐없이 시험할 수 있다. 실제 제품 개발에서도 가능한 상황을 체계적으로 나누어 시험하는 것이 중요하다.

예제 1.2 우산 챙김 알림 장치

현관에 두는 작은 장치를 만든다. 비가 오는데 우산 없이 나가려는 사람에게만 알림을 준다.

입력 R: 비가 온다 O: 외출할 예정이다 U: 우산을 가지고 있다

출력 A: 알림이 울린다

자연어로 쓴 작동 조건은 다음과 같다.

비가 오고, 외출할 예정이며, 우산을 가지고 있지 않을 때에만 알린다.

이를 논리식으로 옮기면 다음과 같다.

$$A = R \wedge O \wedge \neg U$$

아래 진리표의 빈칸을 모두 채우시오.

R	O	U	$\neg U$	$A = R \wedge O \wedge \neg U$
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

연습문제 1.1 진리표 검토하기

완성한 진리표를 표가 아니라 제품이라고 생각하며 다음을 확인한다.

- (1) 비가 오지 않는데 경고하는 행이 있는가?

- (2) 외출하지 않는데 경고하는 행이 있는가?
- (3) 이미 우산을 가지고 있는데 경고하는 행이 있는가?
- (4) 경고가 꼭 필요한 상황인데 경고하지 않는 행이 있는가?

네 질문에 모두 “없다”라고 답할 수 있어야 판단 규칙이 완성된 것이다. A 가 1인 행이 정확히 하나뿐인지도 확인하자.

1.5. 불 함수는 장치의 판단 규칙이다

지금까지 쓴 $\wedge, \vee, \neg$ 는 모두 함수로 볼 수 있다. $\wedge$ 는 두 값을 받아 한 값을 내놓고, $\neg$ 는 한 값을 받아 한 값을 내놓는다. 발명품의 판단 규칙 전체도 마찬가지다.

여러 센서 입력을 받아 출력 장치의 상태를 결정하는 판단 규칙을 불 함수라고 한다.

정의 4 (불 함수). 각 변수의 값이 0 또는 1인 n 개의 변수를 입력받아 0 또는 1을 출력하는 함수

$$f: \{0,1\}^n \rightarrow \{0,1\}$$

를 불 함수(Boolean function)라고 한다.

입력이 P, Q, R 인 불 함수 $f(P, Q, R)$ 을 정의한다는 것은, 8개의 행으로 이루어진 진리표의 마지막 열을 채우는 것과 같다. 우산 알림 장치의 판단 규칙도 $f(R, O, U) = R \wedge O \wedge \neg U$ 라는 불 함수 하나다.

알아두기. 3입력 진리표의 마지막 열에는 8개의 칸이 있고, 각 칸에 0 또는 1을 넣을 수 있다. 따라서 3입력 불 함수는 모두 $2^8 = 256$ 가지가 있다. 센서 세 개만 있어도 서로 다른 판단 규칙 256가지를 설계할 수 있다는 뜻이다. 그중 우리 발명품에 맞는 것을 골라내는 일이 설계다.

1.6. 진리표에서 논리식 만들기

지금까지는 자연어 조건을 논리식으로 옮겼다. 반대 방향도 필요하다. 원하는 작동을 진리표로 먼저 정하고, 그 진리표를 만족하는 논리식을 구성하는 것이다. 방법은 간단하다.

단계 1. 출력이 1인 행을 모두 찾는다.

단계 2. 각 행에 대하여, 그 행에서만 1이 되는 $\wedge$ 식을 만든다. 값이 1인 입력은 그대로 쓰고, 값이 0인 입력은 $\neg$ 를 붙인다.

단계 3. 만든 식들을 $\vee$ 로 연결한다.

예제 1.3 스마트 사물함 출입 제어

학생 카드로 열리는 사물함을 설계한다.

입력 C : 학생 카드가 인식되었다 T : 사용 가능한 시간이다 E : 교사가 긴급 승인하였다

출력 O : 사물함이 열린다

설계자가 처음 제안한 판단 규칙은 다음과 같다.

$$O = (C \wedge T) \vee E$$

(1) 아래 진리표를 완성하시오.

C	T	E	$C \wedge T$	$O = (C \wedge T) \vee E$
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

(2) 출력이 1인 행을 모두 찾고, 위의 세 단계로 논리식을 구성하시오.

(3) (2)에서 만든 식과 $(C \wedge T) \vee E$ 가 모든 행에서 같은 값을 갖는지 확인하시오.

(4) 카드가 없어도 E 만으로 사물함이 열린다. 이 설계에서 생길 수 있는 문제를 사용자와 보안의 관점에서 각각 하나씩 쓰시오.

(5) 위 문제를 줄이는 더 안전한 조건식을 제안하시오. 정답은 하나가 아니다. 제안한 식이 “정말 급할 때 열린다”와 “권한 없이 열리지 않는다”를 동시에 만족하는지 진리표로 확인하시오.

설계 관점. E 처럼 다른 모든 조건을 뛰어넘는 입력을 우회 조건이라 한다. 우회 조건은 응급 상황에 꼭 필요하지만, 그 하나가 틀리면 나머지 설계가 아무 역할도 하지 못한다. 실제 제품에서는 우회 조건에 “기록을 남긴다”, “두 사람이 동시에 승인해야 한다”와 같은 장치를 덧붙인다. E 를 $E_1 \wedge E_2$ 로 바꾸는 것은 바로 이런 발상이다.

1.7. 같은 기능, 다른 설계

두 논리식이 모든 입력에서 같은 값을 가지면, 그 두 식은 같은 기능을 하는 서로 다른 설계다.

정의 5 (논리적 동치). 두 논리식이 모든 입력 조합에서 항상 같은 값을 가질 때, 두 식은 논리적으로 동치(logically equivalent)라 하고 $\equiv$ 로 나타낸다.²

예제 1.4 같은 기능, 다른 설계

과학실의 위험 신호를 두 가지 방식으로 설계했다. 입력은 G (가스가 감지되었다)와 H (온도가 지나치게 높다)이고, 출력은 W (위험 신호를 낸다)이다.

$$\text{설계 A: } W = G \vee H \quad \text{설계 B: } W = \neg(\neg G \wedge \neg H)$$

²기호 $\equiv$ 는 “무엇과 무엇을 같은 것으로 볼 것인가”를 나타낸다. $\triangle ABC \equiv \triangle DEF$ 가 도형을 같게 보는 것이라면, 여기서는 논리식을 같게 보는 것이다.

설계 B는 “가스도 감지되지 않고 온도도 높지 않은 상태가 아니다”를 뜻한다. 아래 표를 채워 두 설계가 모든 입력에서 같은 결과를 내는지 확인하시오.

G	H	$G \vee H$	$\neg G$	$\neg H$	$\neg G \wedge \neg H$	$\neg(\neg G \wedge \neg H)$
0	0					
0	1					
1	0					
1	1					

두 설계가 동치임을 확인했으므로 다음이 성립한다.

$$G \vee H \equiv \neg(\neg G \wedge \neg H)$$

같은 관계는 어떤 명제 P, Q 에 대해서도 성립하며, 이를 드모르간 법칙(*De Morgan's law*)이라고 한다.

$$P \vee Q \equiv \neg(\neg P \wedge \neg Q) \quad P \wedge Q \equiv \neg(\neg P \vee \neg Q)$$

이 식은 $\vee$ 가 들어간 모든 식을 $\neg$ 와 $\wedge$ 만으로 다시 쓸 수 있다는 뜻이다. 수학에서는 “연산자를 줄일 수 있다”는 결과이고, 발명에서는 다음을 뜻한다.

수학적 사실	발명 설계에서의 의미
두 식이 동치다	같은 기능을 서로 다른 부품 구성으로 만들 수 있다
쓰는 연산자가 다르다	필요한 부품의 종류가 달라진다
식의 길이가 다르다	필요한 부품의 개수와 비용이 달라진다
구조가 다르다	조립과 수리의 편의성이 달라진다

같은 기능이라도 설계 방식은 여러 가지다. 어느 것이 더 좋은 설계인지는 부품 수, 부품 종류, 비용, 수리 편의성을 함께 보고 판단해야 한다. 이 비교는 제2장에서 회로로 직접 확인한다.

1.8. 두 상태가 다를 때만: XOR

지금까지의 OR는 둘 다 참인 경우도 포함했다. 반면 “정확히 하나만 참”일 때만 작동해야 하는 장치도 있다.

정의 6 (Xor). 배타적 논리합(*exclusive or*)은 두 입력 중 정확히 하나만 1일 때 1이 되는 연산이며 $P \oplus Q$ 로 쓴다.

P	Q	$P \oplus Q$
0	0	0
0	1	1
1	0	1
1	1	0

장치 관점. 같은 값을 재야 하는 센서를 두 개 달아 두고, 두 센서의 상태가 서로 다를 때 경고하는 센서 불일치 감지 장치를 만들 수 있다. 두 센서가 모두 “정상”이거나 모두 “위험”이면 문제가 없지만, 서로 다른 값을 가리키면 둘 중 하나가 고장 났다는 뜻이므로 점검이 필요하다. 이것이 경고 = $S_1 \oplus S_2$ 이다.

도전 문제 1.2 XOR도 NOT과 AND로 만들 수 있는가 [선택]

XOR가 진짜로 새로운 연산인지 확인해 보자.

- (1) $P \oplus Q$ 의 값이 1이 되는 입력 (P, Q) 를 모두 구하시오.
- (2) (1)의 각 입력에 대하여 그 입력에서만 1이 되는 식을 $\neg$ 와 $\wedge$ 만으로 만드시오.
- (3) (2)의 식들을 $\vee$ 로 연결하여 $P \oplus Q$ 를 나타내시오.
- (4) 드모르간 법칙 $A \vee B \equiv \neg(\neg A \wedge \neg B)$ 를 이용하여 (3)의 $\vee$ 를 없애고, $P \oplus Q$ 를 $\neg$ 와 $\wedge$ 만으로 나타내시오.

이 과정은 XOR만의 특별한 성질이 아니다. 어떤 진리표가 주어져도 같은 방법으로 $\neg$ 와 $\wedge$ 만으로 이루어진 식을 만들 수 있다. 이처럼 모든 불 함수를 표현할 수 있는 연산자 집합을 함수적으로 완전한(functionally complete) 집합이라 하며, 따라서 $\{\neg, \wedge\}$ 는 함수적으로 완전하다.

연습문제 1.3 진리표에서 논리식 구성

아래 불 함수 $f(P, Q, R)$ 과 동치인 논리식을 $\neg, \wedge, \vee$ 를 사용하여 나타내시오. 이어서 $\neg$ 와 $\wedge$ 만으로도 나타내시오.

P	Q	R	$f(P, Q, R)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

1.9. 발명 설계 미션

이제 우리 팀의 발명품을 정한다. 아래 양식을 채우면 제2장에서 회로로, 제3~4장에서 Python 프로그램으로 이어진다.

설계 문제 1.4 우리 팀 발명품의 작동 조건 정하기

해결하려는 문제

사용자

입력 조건 2 ~ 3개

출력 행동 1 ~ 2개

자연어 작동 조건

논리식

점검. 입력은 모두 “예/아니오”로 답할 수 있는가? 출력은 장치가 실제로 할 수 있는 행동인가? 자연어 조건과 논리식이 정확히 같은 뜻인가?

설계 문제 1.5 진리표 작성

입력이 세 개라면 아래 표의 여덟 행을 모두 채운다. 입력이 두 개라면 위의 네 행만 사용한다. 출력이 두 개인 팀은 마지막 열을 두 개로 나누어 쓴다.

입력 1	입력 2	입력 3	출력
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

점검. 출력이 1인 행을 소리 내어 읽어 보고, 정말 그 상황에서 장치가 작동해야 하는지 팀원과 확인한다. 출력이 0인 행도 같은 방법으로 확인한다. 이 표는 2일차에 Python 테스트 코드의 `TEST_CASES`가 된다.

2. 논리게이트: 판단 회로에서 이진 덧셈까지

오늘의 목표. 제1장에서 우리는 발명품의 작동 조건을 논리식으로 적었다. 그런데 논리식은 종이 위의 식일 뿐이다. 전기로 움직이는 장치가 이 식대로 판단하게 하려면 무엇이 필요할까? 이 장에서는 논리식을 판단 회로로 바꾸고, 같은 기능을 더 간단히 만드는 방법을 살펴본다. 그리고 마지막에는 참과 거짓만 다루던 논리게이트가 실제 이진수의 덧셈까지 해낸다는 사실에 이른다.

2.1. 불 대수에서 전기 회로로

논리를 대수식으로 쓸 수 있다면, 그 연산을 수행하는 기계도 만들 수 있다. 이 생각을 실제로 증명한 사람은 클로드 섀넌(Claude Shannon)이다. 그는 1937년 MIT 석사 논문에서 모든 불 대수 연산을 전기 스위치의 조합으로 구현할 수 있음을 보였다(Petzold, 2000). 이것이 오늘날 모든 컴퓨터와 스마트 기기 회로 설계의 출발점이다.

일정한 시간 간격마다 전선에 전기 신호가 도착하거나 도착하지 않는 상황을 생각하자. 신호가 도착하면 1, 도착하지 않으면 0으로 읽는다. 제1장에서 정한 0과 1이 이제 실제 전기 신호가 된다.

논리게이트는 입력 신호를 받아 정해진 논리 규칙에 따라 출력 신호를 만드는 작은 판단 장치다.

정의 7 (논리게이트). 불 함수

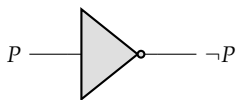
$$f : \{0,1\}^n \rightarrow \{0,1\}$$

를 물리적으로 구현한 전기 회로를 게이트(gate)라고 한다.

즉 게이트는 제1장의 불 함수를 손으로 만질 수 있는 부품으로 바꾼 것이다. 논리식의 연산자 하나가 게이트 하나에 대응한다.

2.2. NOT, AND, OR 게이트

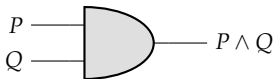
기본 게이트는 각각 정해진 기호로 그린다. 회로는 왼쪽에서 오른쪽으로 읽는다. 왼쪽 선으로 입력이 들어오고 오른쪽 선으로 출력이 나간다.



P	¬P
0	1
1	0

NOT 게이트

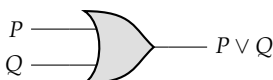
입력 상태를 반대로 해석한다.
“우산이 없을 때”를 만드는 부품이다.



P	Q	P ∧ Q
0	0	0
0	1	0
1	0	0
1	1	1

AND 게이트

두 조건을 모두 만족할 때만 작동한다.
꼭 필요할 때만 켜서 낭비를 줄인다.



P	Q	P ∨ Q
0	0	0
0	1	1
1	0	1
1	1	1

OR 게이트

위험 조건 중 하나라도 감지되면 작동한다.
안전 장치의 기본 부품이다.

그림 2.1: 기본 게이트의 기호, 진리표, 장치 관점의 해석

하나의 신호를 두 곳에 보내려면 선을 갈라 주어야 한다. 이렇게 한 신호를 여러 개로 복제하는 것을 팬아웃(*fan-out*)이라 하고, 회로도에서는 갈라지는 지점에 점을 찍어 나타낸다. 점이 없는 교차는 두 선이 그냥 지나가는 것이며 연결되어 있지 않다.

2.3. 논리식을 회로로 바꾸기

논리식을 회로로 옮기는 순서는 다음과 같다.

- 단계 1. 입력마다 왼쪽에 선을 하나씩 그리고 이름을 붙인다.
- 단계 2. $\neg$ 가 붙은 입력에는 먼저 NOT 게이트를 놓는다.
- 단계 3. 괄호 안의 연산부터 게이트로 바꾼다.
- 단계 4. 게이트에서 나온 선에 그 선이 나타내는 논리식을 적어 둔다.
- 단계 5. 마지막 연산의 출력을 출력 장치로 연결한다.

우산 챙김 알림 장치의 판단 규칙 $A = R \wedge O \wedge \neg U$ 를 이 순서대로 옮기면 그림 2.2이 된다. U 에는 NOT 게이트를 먼저 놓고, AND 게이트 두 개로 세 조건을 묶는다.

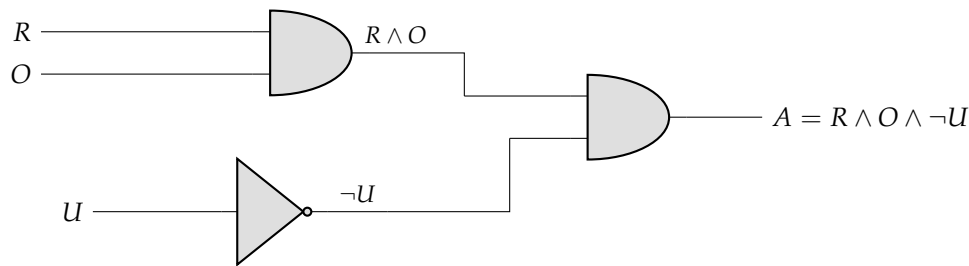


그림 2.2: 우산 챙김 알림 장치의 판단 회로

설계 문제 2.1 우리 팀 논리식을 회로로

제1장 설계 문제 1.4에서 적은 논리식을 위의 다섯 단계에 따라 회로로 그리시오. 각 선에 그 선이 나타내는 논리식을 반드시 적는다. 그런 뒤 다음을 확인한다.

- 사용한 게이트의 개수는 몇 개인가?
- 사용한 게이트의 종류는 몇 가지인가?
- 진리표에서 출력이 1인 행 하나를 골라, 그 입력을 회로에 넣고 선을 따라가며 출력이 정말 1이 되는지 확인하시오.

2.4. 스마트 과학실 안전관리 시스템

이제 하나의 센서 묶음으로 두 개의 출력 장치를 제어하는 설계를 살펴본다. 이 예제는 이 강의 전체에서 가장 중요한 설계 원리를 담고 있다. 다만 이 시스템은 논리식을 익히기 위한 교육용 가상 모델이다. 실제 실험실 안전 설비는 센서 이중화, 정전 대비, 오작동 시의 안전 측 동작 등 훨씬 많은 요구 조건을 만족해야 하며, 여기서 만든 회로를 그대로 쓸 수는 없다.

입력 G: 가스 또는 유해 물질이 감지되었다 H: 온도가 지나치게 높다 P: 실험실 안에 사람이 있다

출력 1 A: 대피 정보가 울린다

출력 2 F: 환풍기가 작동한다

먼저 “위험”을 하나의 중간 신호로 정의한다. 가스든 고온이든 하나만 감지되어도 위험이므로 OR를 쓴다.

$$\text{위험} = G \vee H$$

그다음 두 출력을 각각 설계한다.

$$A = (G \vee H) \wedge P \quad F = G \vee H$$

설계 관점. 같은 센서를 쓰는데 왜 두 출력의 조건식이 다른가?

- 대피 경보는 사람에게 위험을 알리는 출력이다. 아무도 없는 실험실에서 경보를 울리는 것은 의미가 없고, 오히려 경보에 대한 신뢰만 떨어뜨린다. 그래서 $\wedge P$ 가 붙는다.
- 환풍기는 위험 자체를 제거하는 출력이다. 사람이 없을 때도 가스는 계속 쌓이므로, 사람이 없어도 작동해야 한다. 그래서 P 가 조건에 들어가지 않는다.

같은 입력을 쓰더라도 출력의 목적이 다르면 논리식이 달라진다. “사용자에게 무엇을 알리는가”와 “문제를 어떻게 없애는가”를 구분하는 것이 사용자 중심 설계이자 안전 설계다.

예제 2.1 두 출력의 진리표와 회로

(1) 아래 진리표를 완성하시오.

G	H	P	$G \vee H$	$A = (G \vee H) \wedge P$	$F = G \vee H$
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

(2) A 가 1인 행은 몇 개이고, F 가 1인 행은 몇 개인가? 왜 개수가 다른지 설명하시오.

(3) $A = 0$ 이면서 $F = 1$ 인 행을 모두 찾고, 그 상황을 말로 설명하시오.

(4) 두 출력을 함께 만드는 회로를 그리시오. OR 게이트의 출력을 팬아웃으로 갈라 쓰면 게이트 하나를 아낄 수 있다.

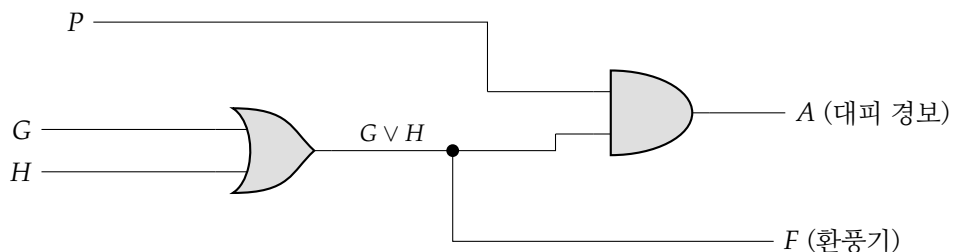


그림 2.3: 스마트 과학실 안전관리 시스템의 판단 회로. OR 게이트의 출력을 팬아웃으로 갈라 두 출력에 함께 사용한다.

2.5. 같은 기능을 더 경제적으로

제1장에서 두 논리식이 동치일 수 있음을 보았다. 회로로 옮기면 그 차이가 눈에 보인다. 실제 부품은 한 칩 안에 게이트가 여러 개 들어 있기도 하고 배선과 전원도 함께 필요하므로 게이트 하나가 곧 부품 하나인 것은 아니다. 다만 이 수업의 단순화된 회로 모델에서는 게이트 수를 부품 수의 한 지표로 볼 수 있으며, 식이 짧아지면 대체로 다음이 따라온다.

설계 변경	발명적 효과
게이트 수 감소	부품 수와 비용이 줄어든다
게이트 종류 감소	사야 할 부품 종류가 줄고 재고 관리가 쉬워진다
연결선 감소	조립 중 연결 오류가 줄어든다
불필요한 게이트 제거	소비 전력이 줄어 배터리가 오래간다
구조 단순화	고장 지점을 찾기 쉬워 수리가 편해진다

그런데 식을 짧게 만드는 데 복잡한 기법이 늘 필요한 것은 아니다. 다음 세 가지 동치만 알아도 학생 설계에서 나오는 낭비의 대부분을 없앨 수 있다.

$$P \wedge P \equiv P \quad P \vee P \equiv P \quad P \vee (P \wedge Q) \equiv P$$

동치	발명 설계에서의 뜻
$P \wedge P \equiv P$	같은 조건을 두 번 검사하고 있다. 센서 하나와 AND 게이트 하나가 낭비다.
$P \vee P \equiv P$	같은 위험을 두 경로로 검사하고 있다. 한 경로면 충분하다.
$P \vee (P \wedge Q) \equiv P$	Q가 무엇이든 결과가 바뀌지 않는다. Q 센서와 AND 게이트를 모두 뺄 수 있다.

예제 2.2 낭비를 찾아내기

어느 모뎀이 야간 보행 안전등을 다음과 같이 설계했다. D 는 어둡다, W 는 보행자가 지나간다는 뜻이다.

$$L = (D \wedge W) \vee D$$

- (1) 진리표를 만들어 이 식이 $L = D$ 와 동치임을 확인하십시오.
- (2) 이 설계는 보행자 센서 W 를 달았지만, 실제로는 무엇을 하고 있는가?
- (3) 게이트를 몇 개 줄일 수 있는가?
- (4) 이 모뎀이 원래 만들고 싶었던 장치는 $L = D \wedge W$ 였을 것이다. $\vee$ 를 $\wedge$ 로 바꾸면 진리표가 어떻게 달라지는가? 전력 소비 관점에서 어느 설계가 더 좋은가?

2.6. NAND와 부품 표준화 [심화]

이 절은 수업의 흐름상 선택 내용이다. 시간이 부족하면 건너뛰고 바로 제2.7절로 넘어가도 다음 내용을 이해하는 데 지장이 없다.

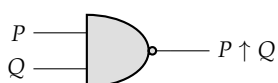
지금까지 NOT, AND, OR 세 종류의 부품을 썼다. 그런데 실제 부품을 사려면 종류마다 따로 주문하고, 따로 보관하고, 고장 나면 종류에 맞게 갈아 끼워야 한다. 그래서 이런 질문이 생긴다.

한 종류의 부품만 사용하여 여러 기능을 만들 수 있을까?

놀랍게도 가능하다. 그 부품이 NAND다.

정의 8 (Nand). *Nand*는 *not*과 *and*를 결합한 혼성어(portmanteau)이며 기호 $\uparrow$ 로 나타낸다.³

$$P \uparrow Q = \neg(P \wedge Q)$$



P	Q	$P \uparrow Q$
0	0	1
0	1	1
1	0	1
1	1	0

NAND 게이트
두 조건이 모두 만족될 때만 출력이 꺼진다.
AND 게이트 뒤에 NOT을 붙인 것과 같다.

그림 2.4: NAND 게이트의 기호와 진리표

NAND 하나로 NOT, AND, OR를 모두 만들 수 있다. $\{\neg, \wedge\}$ 가 함수적으로 완전하다는 것은 이미 알고 있으므로, NOT과 AND를 NAND로 만들 수 있음을 보이면 충분하다.

$$\neg P \equiv P \uparrow P$$

$$P \wedge Q \equiv (P \uparrow Q) \uparrow (P \uparrow Q)$$

$$P \vee Q \equiv (P \uparrow P) \uparrow (Q \uparrow Q)$$

첫 번째 식은 팬아웃으로 P 를 두 갈래로 나누어 NAND의 두 입력에 함께 넣는다는 뜻이다. 그림 2.5이 이 회로다.

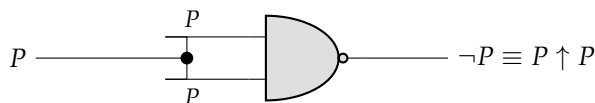


그림 2.5: NAND 하나와 팬아웃으로 만든 NOT 게이트

예제 2.3 NAND만으로 AND와 OR 만들기

그림 2.5을 참고하여 다음을 NAND 게이트와 팬아웃만으로 회로로 그리시오. 각 선에 논리식을 적는다.

- (1) $P \wedge Q$
- (2) $P \vee Q$
- (3) (1)과 (2)에 사용한 NAND 게이트는 각각 몇 개인가? 기본 게이트로 만들 때보다 부품 개수는 늘어났는가, 줄어들었는가?

이렇게 어떤 불 함수든 NAND 게이트 하나의 조합으로 구현할 수 있다. 게이트에 대해서는 “함수적으로 완전하다” 대신 범용적(*universal*)이라는 말을 쓰므로, NAND는 범용 게이트(*universal gate*)이다.

설계 관점. 예제 2.3에서 확인했듯이 NAND만 쓰면 게이트 개수는 늘어난다. 그런데도 실제 산업에서 NAND를 많이 쓰는 이유는 다음과 같다.

- 부품 종류가 하나이므로 대량으로 싸게 살 수 있다.
- 창고에 한 종류만 보관하면 되므로 재고 관리가 단순하다.

³1913년 헨리 셰퍼(Henry M. Sheffer)가 $\uparrow$ 대신 $|$ 를 사용하였으므로 이 연산을 셰퍼 스트로크라고도 한다. 찰스 샌더스 퍼스(Charles S. Peirce)도 19세기 후반에 같은 사실을 알고 있었으나 훨씬 뒤에 출판되었다.

- 어느 자리가 고장 나도 같은 부품으로 교체할 수 있다.
- 같은 부품을 반복해서 쓰므로 조립 과정을 표준화하기 쉽다.

“부품 개수를 줄이는 최적화”와 “부품 종류를 줄이는 표준화”는 서로 다른 목표이고, 때로는 서로 충돌한다. 어느 쪽을 택할지는 만들려는 제품의 상황이 결정한다.

도전 문제 2.2 우리 팀 회로를 NAND만으로 [선택]

설계 문제 2.1에서 그린 우리 팀 회로를 NAND 게이트와 팬아웃만으로 다시 그리시오. 원래 회로의 게이트 수, 게이트 종류 수와 NAND 회로의 게이트 수를 표로 비교하고, 우리 발명품에는 어느 설계가 더 적합한지 이유를 들어 쓰시오.

2.7. 논리게이트로 이진수를 더한다

지금까지 게이트는 “조건이 맞는가”를 판단했다. 그런데 같은 게이트로 계산도 할 수 있다. 여기서부터가 이 강의의 수학적 절정이다.

한 자리 이진수의 덧셈. 한 자리 이진수 두 개를 더해 보자. 십진수처럼 자리와 올림을 나누어 적는다.

$$0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 0 = 1, \quad 1 + 1 = 10_{(2)}$$

앞의 세 경우는 한 자리로 끝나지만, $1 + 1$ 은 두 자리가 된다. $1 + 1$ 은 십진수로 2이고, 이진수로는 $10_{(2)}$ 이므로 합의 자리는 0, 뒷자리로 올라가는 올림은 1이다. 네 경우를 표로 정리하면 다음과 같다.

A	B	Sum (합의 자리)	Carry (올림)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

예제 2.4 합과 올림을 논리식으로

위 표의 Sum 열과 Carry 열을 각각 제1장에서 배운 논리 연산과 비교하시오.

- (1) Sum 열과 진리표가 완전히 같은 연산은 무엇인가?
- (2) Carry 열과 진리표가 완전히 같은 연산은 무엇인가?
- (3) 두 결과를 논리식으로 쓰시오.

Sum 열은 두 입력이 서로 다를 때만 1이므로 XOR이고, Carry 열은 두 입력이 모두 1일 때만 1이므로 AND다.

$$\text{Sum} = A \oplus B \quad \text{Carry} = A \wedge B$$

논리 연산이 덧셈이 되었다. 사실 이것은 우연이 아니다. 0을 짝수, 1을 홀수로 보면 짝수+짝수=짝수, 짝수+홀수=홀수, 홀수+홀수=짝수가 되어 XOR는 정확히 모듈로 2 덧셈이 된다. 기호 $\oplus$ 안에 더하기 기호가 들어 있는 이유가 바로 이것이다.

반가산기. 위의 두 식을 회로로 만든 것이 반가산기(*half-adder*)다.

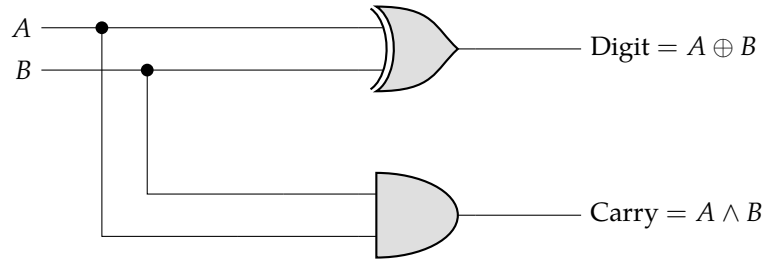


그림 2.6: 반가산기(*half-adder*) 회로. 점이 찍힌 지점은 팬아웃이고, 점이 없는 교차는 연결되어 있지 않다.

왜 “반”가산기인가. 여러 자리 이진수를 세로셈으로 더할 때를 떠올려 보자. 오른쪽 자리에서 올림이 발생하면 왼쪽 자리는 그 올림까지 함께 더해야 한다. 그런데 반가산기의 입력은 A 와 B 둘뿐이라, 오른쪽에서 넘어온 올림을 받을 자리가 없다. 절반만 하는 셈이므로 “반”가산기다.

그렇다면 입력이 A, B 두 개가 아니라 A, B, C_{in} 세 개가 되어야 하지 않을까? 이렇게 이전 자리의 올림까지 받아 더하는 회로를 전가산기(*full adder*)라고 한다.

연습문제 2.3 전가산기의 진리표

A, B, C_{in} 을 모두 더한 값을 두 자리 이진수 $(\text{CoutSum})_{(2)}$ 로 나타낸다고 하자. 예를 들어 $1 + 1 + 1 = 3 = 11_{(2)}$ 이므로 $\text{Sum} = 1, \text{Cout} = 1$ 이다. 아래 표를 완성하시오.

A	B	C_{in}	$A + B + C_{in}$	Sum	Cout
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

완성한 뒤 다음을 확인하시오.

- (1) Sum이 1인 행은 세 입력 중 1의 개수가 몇 개일 때인가?
- (2) Cout이 1인 행은 세 입력 중 1의 개수가 몇 개일 때인가?

반가산기 두 개를 이어 붙이면 전가산기가 된다. 전가산기를 처음부터 새로 설계할 필요는 없다. $A + B + C_{in}$ 을 한꺼번에 더하는 대신 두 번에 나누어 더하면 된다.

먼저 A 와 B 를 반가산기로 더한다.

$$S_1 = A \oplus B, \quad C_1 = A \wedge B$$

그다음 그 결과 S_1 에 이전 자리의 올림 C_{in} 을 또 하나의 반가산기로 더한다.

$$\text{Sum} = S_1 \oplus C_{in}, \quad C_2 = S_1 \wedge C_{in}$$

올림은 두 번의 덧셈 중 어느 쪽에서든 발생할 수 있고, 두 곳에서 동시에 발생하는 일은 없다. 따라서 최종 올림은 OR

로 모으면 된다.

$$C_{out} = C_1 \vee C_2$$

정리하면 전가산기의 두 출력은 다음과 같다.

$$Sum = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \wedge B) \vee ((A \oplus B) \wedge C_{in})$$

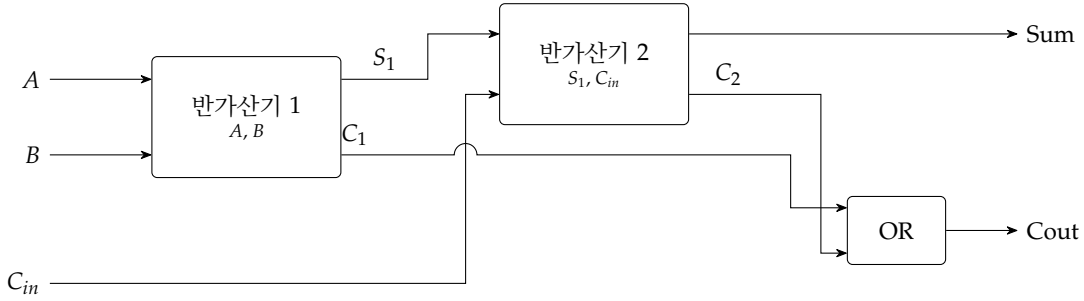


그림 2.7: 전가산기의 구조. 반가산기 두 개와 OR 게이트 하나로 이루어진다. C_1 선이 반원으로 넘어가는 곳은 C_{in} 선과 연결되어 있지 않다는 표시다. 게이트 단위의 회로도는 부록 D.4에 있다.

검산. 전가산기가 정말 덧셈을 하고 있는지는 다음 식으로 확인할 수 있다.

$$A + B + C_{in} = Sum + 2 C_{out}$$

왼쪽은 보통의 정수 덧셈이고, 오른쪽은 두 자리 이진수 $(C_{out}Sum)_{(2)}$ 를 십진수로 읽은 것이다. 연습문제 2.3의 여덟 행 모두에서 이 식이 성립하는지 확인해 보자. 성립한다면 우리가 만든 것은 판단 회로가 아니라 진짜 계산기다.

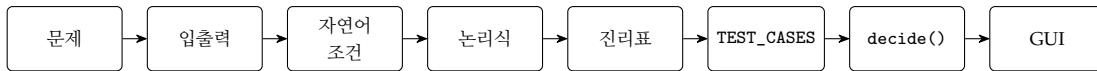
여기서 컴퓨터까지. 전가산기 한 개는 한 자리를 더한다. 전가산기를 여러 개 옆으로 이어 붙이고, 각 자리의 C_{out} 을 왼쪽 자리의 C_{in} 에 연결하면 여러 자리 이진수의 덧셈 회로가 된다. 그리고 모든 게이트는 NAND로 바꿀 수 있으므로, 결국 NAND와 팬아웃만으로 컴퓨터 전체를 만들 수 있다(Petzold, 2000). 참과 거짓을 다루던 불 대수가 여기까지 온 것이다.

2.8. 2일차 프로젝트 미리 보기

1일차에 배운 수학은 모두 같다. 오늘은 아직 모듈이나 트랙을 정하지 않는다. 각자 다음 두 트랙을 읽고 관심 있는 문제나 아이디어를 생각해 둔다. 2일차에 개인 아이디어를 공유하고 팀을 만든 뒤 한 트랙을 택한다.

	A 트랙: 스마트 발명품	B 트랙: 이진 계산 장치
만드는 것	생활 속 문제를 해결하는 판단 장치	한 자리 이진수를 더하는 전가산기
입력	센서 2 ~ 3개 (사람 있음, 어두움, ...)	비트 A, B, C_{in}
출력	출력 장치 1 ~ 2개 (조명, 경보, ...)	Sum, C_{out}
판단 규칙	우리 팀이 직접 설계한 논리식	1일차에 유도한 전가산기 식
잘 맞는 모듈	문제 상황과 사용자에게 관심이 있는 모듈	수학과 컴퓨터 구조에 관심이 있는 모듈

두 트랙은 겉모습만 다를 뿐 하는 일은 완전히 같다. 어느 쪽을 택하든 다음 과정을 그대로 거친다.



A 트랙은 “센서 입력 → 논리식 → 출력 장치”이고, B 트랙은 “비트 입력 → 논리식 → Sum/Carry”다. 같은 수학 구조를 서로 다른 대상에 적용하는 것뿐이다.

아래 설계서는 2일차 팀 활동에서 사용한다. 오늘은 팀 설계서를 채우지 않고, 제3장 마지막에 개인 아이디어 카드만 준비한다. 2일차에 아이디어를 공유하고 팀을 만든 뒤 A 트랙은 설계 문제 2.4, B 트랙은 설계 문제 2.5를 작성하고, 두 트랙 모두 설계 문제 2.6의 회로 설계도를 확인한다. 팀이 문제와 입출력, 조건, 논리식과 최종 진리표에 합의한 뒤 코드를 시작한다.

설계 문제 2.4 (A 트랙) 모둠 발명 설계서

모둠 이름 / 발명품 이름	
해결하려는 문제	
사용자	
입력 조건 2 ~ 3 개 (기호와 뜻)	
출력 행동 1 ~ 2 개 (기호와 뜻)	
자연어 작동 조건	
논리식 (출력마다 하나)	
사용한 게이트 수 / 종류	
예상 오작동	
개선 아이디어	

설계 문제 2.5 (B 트랙) 전가산기 설계 확인표

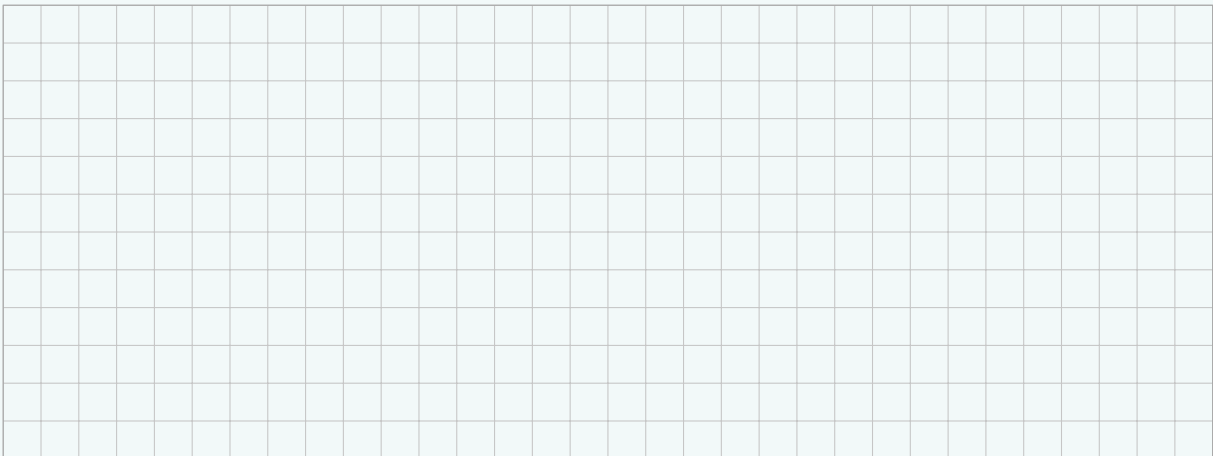
B 트랙은 만들 장치가 이미 정해져 있으므로, 설계서를 새로 쓰는 대신 아래 항목을 하나씩 확인한다.

모듈 이름	
입력 세 개의 뜻	$A:$ $B:$ $C_{in}:$
출력 두 개의 뜻	$Sum:$ $Cout:$
Sum 의 논리식	
$Cout$ 의 논리식	
연습문제 2.3의 진리표 여덟 행을 모두 채웠는가	
여덟 행 모두에서 $A + B + C_{in} = Sum + 2 Cout$ 인가	
사용자가 이 장치를 어떻게 쓰는가	

마지막 칸이 중요하다. 계산 장치도 결국 사람이 쓰는 물건이다. 세 비트를 어떻게 입력하고 결과를 어떻게 읽을지 미리 정해 두어야 2일차에 화면을 설계할 수 있다.

설계 문제 2.6 회로 설계도

아래 칸에 우리 팀의 회로를 그린다. A 트랙은 판단 회로를, B 트랙은 전가산기 회로를 게이트 단위로 그린다. 입력 선에 기호를 쓰고, 게이트에서 나오는 선마다 논리식을 적는다. 출력이 두 개인 팀은 두 출력을 함께 그리고, 팬아웃으로 공유할 수 있는 부분이 있는지 표시한다.



예상 오작동 점검. 진리표를 한 행씩 읽으며 다음을 확인한다. A 트랙은 작동해야 하는데 작동하지 않는 행, 그리고 불필요하게 작동하는 행이 있는지 본다. B 트랙은 진리표와 다른 Sum , $Cout$ 이 나오는 행이 있는지 보고, 특히 올림이 생기는 행과 생기지 않는 행을 모두 확인했는지 점검한다. 찾은 내용을 설계서의 예상 오작동 칸에 적는다.

3. Python 논리 실험실

1일차 — 개인 활동. 제1장과 제2장에서 판단 규칙과 가산기의 논리식을 세우고 회로로 그렸다. 그런데 게이트를 실제로 사서 조립하려면 시간과 비용이 많이 든다. 설계가 옳은지 먼저 확인할 방법이 필요하다. 이 차시에서는 각자 공통 예제의 논리식을 Python 함수로 옮기고, 모든 입력 조합을 자동으로 시험하여 판단 논리에 오작동이 없는지 확인한다. 오늘은 모두 같은 공통 파일을 사용하며, 마지막에는 자신의 발명 아이디어를 짧게 적는다.

시간	활동	내용
0 ~ 8분	환경 확인	Python과 tkinter 실행 확인, True/False 실험
8 ~ 16분	연산자 번역	and, or, not과 $\wedge$, $\vee$, $\neg$ 의 대응
16 ~ 25분	판단 함수	논리식을 decide() 함수로 옮기기
25 ~ 34분	진리표 자동 생성	itertools.product로 8가지 조합 만들기
34 ~ 42분	테스트와 수리	TEST_CASES 확인, 고장 난 함수 고치기
42 ~ 50분	다음 Lab 준비	우산 GUI 실행, 개인 아이디어 카드 작성

3.1. 실습 환경 확인

이 실습은 컴퓨터에 설치된 Python 3만으로 진행한다. 추가로 설치할 프로그램이나 패키지는 없다. 편집기는 IDLE, VS Code 등 익숙한 것을 쓰면 된다.

tkinter는 따로 설치하는 응용 프로그램이 아니라, 창과 버튼이 있는 화면을 만드는 데 쓰는 Python의 표준 GUI 라이브러리다. 다만 설치 환경에 따라 포함되지 않은 경우가 있으므로 `python -m tkinter`로 미리 확인한다.

실습 문제 3.1 실행 환경 확인

터미널(명령 프롬프트)을 열고 다음을 차례로 입력한다.

```
python --version
python -m tkinter
```

첫 명령은 Python 버전을 보여 주고, 둘째 명령은 작은 창을 띄운다. 창이 열리면 실습 준비가 끝난 것이다.

Windows에서 python 명령이 인식되지 않으면 py를 사용한다.

```
py --version
py -m tkinter
```

이어서 code 폴더의 00_environment_check.py를 실행한다. 터미널에 다음과 같이 출력되고 작은 창이 열린다.

```
=====
실습 환경 확인
=====
Python 버전 : 3.13.2
tkinter 버전: 8.6
외부 패키지는 설치하지 않아도 된다.
=====
```

3.2. 불 대수와 Python의 번역

Python에는 불 대수가 그대로 들어 있다. 0과 1 대신 False와 True라는 이름을 쓰는 것뿐이다. Python은 이 세 연산을 불 연산(Boolean operation)이라고 부른다([Python Software Foundation, 2024a](#)).

수학 표기	Python 표기	읽는 법
$P \wedge Q$	P and Q	P이고 Q이다
$P \vee Q$	P or Q	P이거나 Q이다
$\neg P$	not P	P가 아니다
참, 거짓	True, False	참, 거짓
1, 0	int(True), int(False)	진리값을 0과 1로 나타낸 것

int(True)가 1이고 int(False)가 0이라는 점은 진리표를 출력할 때 요긴하다. True, False보다 1, 0이 표로 읽기 훨씬 쉽다.

주의. Python에는 &, |, ~ 기호도 있다. 이들은 비트 연산자로, 논리 판단이 아니라 정수의 각 비트를 다루는 데 쓰인다. 우연히 비슷한 결과가 나오는 경우도 있지만 규칙이 다르고, 연산 순서도 달라서 조건식이 조용히 틀리게 계산될 수 있다. XOR에 해당하는 ^도 마찬가지다. 이번 수업의 논리식은 반드시 and, or, not으로만 쓴다. XOR가 필요하면 제1장에서 유도한 대로 이 세 연산으로 직접 만든다.

실습 문제 3.2 세 가지 연산 확인

01_boolean_practice.py를 실행한다. 파일에는 세 연산을 하나씩 확인하는 부분이 들어 있다.

```

1 def show_operators() -> None:
2     """세 가지 기본 연산을 하나씩 확인한다."""
3     print("[2] 세 가지 기본 연산")
4     print(" True and False =", True and False)
5     print(" True or False =", True or False)
6     print(" not True      =", not True)
7     print(" not False     =", not False)

```

실행하면 다음이 출력된다.

[2] 세 가지 기본 연산

```

True and False = False
True or False  = True
not True       = False
not False      = True

```

[3] 두 입력 진리표

P	Q	not P	P and Q	P or Q
0	0	1	0	0
0	1	1	0	1
1	0	0	0	1
1	1	0	1	1

[4] 실험: 스마트 복도 조명

```

P(사람) = 1 D(어둠) = 0
L = P and D = 0
-> 조명을 켜지 않는다.

```

person_here와 is_dark의 값을 바꾸어 네 가지 경우를 모두 확인하고, 종이에 쓴 진리표와 같은지 비교해 보자.

표로 나온 0과 1이 제1장에서 손으로 채운 진리표와 같은지 비교한다.

실습 문제 3.3 값을 바꾸어 실험하기

같은 파일 아래쪽에는 스마트 복도 조명을 확인하는 실험 구역이 있다.

```
1 # 스마트 복도 조명: L = P and D
2 person_here = True      # P: 사람이 있다
3 is_dark = False         # D: 어둡다
4
5
6 def hallway_light(person: bool, dark: bool) -> bool:
7     """복도 조명의 판단 규칙: 사람이 있고 어두울 때만 켜다."""
8     return person and dark
```

person_here와 is_dark의 값을 True/False로 바꾸어 네 가지 경우를 모두 확인한다. 값을 바꾼 뒤에는 반드시 파일을 저장하고 다시 실행해야 결과가 바뀐다.

확인 질문: return person and dark를 return person or dark로 바꾸면 어떤 경우의 결과가 달라지는가? 그 설계는 전기를 더 쓰는가, 덜 쓰는가?

3.3. 판단 함수 만들기

발명품의 판단 회로를 Python에서는 판단 함수로 만든다. 우산 챙김 알림 장치의 논리식과 함수를 나란히 보자.

$$A = R \wedge O \wedge \neg U$$

```
1 def decide(rain: bool, going_out: bool, has_umbrella: bool) -> bool:
2     """올바른 판단 규칙: 비가 오고, 외출하는데, 우산이 없을 때만 알린다."""
3     return rain and going_out and not has_umbrella
```

대응 관계는 다음과 같다.

논리식	Python
입력 R, O, U	rain, going_out, has_umbrella
$\wedge$	and
$\neg U$	not has_umbrella
출력 A	return으로 돌려주는 값

함수 문법을 깊이 알 필요는 없다. 다음 세 가지만 기억하면 된다.

- 센서의 상태가 함수의 입력이 된다.
- 논리식을 계산한 결과가 함수가 돌려주는 값이 된다.
- decide 함수가 발명품의 판단 회로 역할을 한다.

rain: bool처럼 이름 뒤에 붙은 bool은 “이 값은 참 또는 거짓이다”라고 읽는 사람에게 알려 주는 표시이며, 없어도 프로그램은 똑같이 동작한다.

3.4. Python으로 진리표 생성하기

입력이 세 개면 여덟 가지 조합을 손으로 적어야 했다. Python의 itertools.product를 쓰면 이 조합을 자동으로 만들 수 있다. itertools도 표준 라이브러리이므로 설치할 것이 없다.

```

1 def print_truth_table(rule) -> None:
2     """세 입력의 모든 조합을 만들어 판단 결과를 0과 1로 출력한다."""
3     print(" {} {} {} | A".format(*INPUT_NAMES))
4     print(" -----+----")
5     for rain, going_out, has_umbrella in itertools.product(
6         (False, True), repeat=3
7     ):
8         answer = rule(rain, going_out, has_umbrella)
9         print(
10             " {} {} {} | {}".format(
11                 int(rain), int(going_out), int(has_umbrella), int(answer)
12             )
13         )

```

`itertools.product((False, True), repeat=3)`은 False가 먼저 오는 순서로 조합을 만들어 준다. 그래서 출력되는 진리표의 행 순서가 000,001,010,...,111이 되어 제1장에서 정한 순서와 정확히 같다. `int(rain)`은 True를 1로, False를 0으로 바꾼다.

실습 문제 3.4 진리표 자동 생성

02_truth_table_lab.py를 실행하면 다음이 출력된다.

[1] 올바른 판단 규칙의 진리표

```

R  O  U | A
-----+----
0  0  0 | 0
0  0  1 | 0
0  1  0 | 0
0  1  1 | 0
1  0  0 | 0
1  0  1 | 0
1  1  0 | 1
1  1  1 | 0

```

[2] 올바른 판단 규칙의 제품 테스트

테스트 결과: 8 / 8 통과
모든 입력 조합에서 의도한 대로 작동한다.

이 진리표를 제1장 예제 1.2에서 손으로 채운 진리표와 한 행씩 비교한다. 다른 행이 있으면 둘 중 하나가 틀린 것이다. 어느 쪽이 틀렸는지 찾는다.

3.5. 테스트 코드는 제품 시험표다

여기서 중요한 구분을 하나 해 두자.

종이에 쓴 진리표 제품의 사양이다. 이렇게 작동해야 한다는 약속이다.

Python 테스트 코드 실제 제품 검사다. 만든 것이 약속대로 작동하는지 확인한다.

그래서 순서가 중요하다. 먼저 진리표에 예상 출력을 적고, 그것을 코드로 옮긴다. 코드를 먼저 쓰고 그 결과를 예상 출력이라고 적으면 검사가 아무것도 검사하지 못한다.

```

1 # 제품 시험표: 종이 진리표에서 예상 출력을 옮겨 적은 것
2 # ((비, 외출, 우산), 예상 출력)
3 # =====
4

```

```

5 TEST_CASES = [
6     ((False, False, False), False),
7     ((False, False, True), False),
8     ((False, True, False), False),
9     ((False, True, True), False),
10    ((True, False, False), False),
11    ((True, False, True), False),
12    ((True, True, False), True),
13    ((True, True, True), False),
14 ]

```

각 줄은 “이 입력을 넣으면 이 출력이 나와야 한다”는 뜻이다. 앞의 괄호는 세 입력, 뒤의 값은 예상 출력이다.

```

1 def run_tests(rule=decide) -> bool:
2     """모든 테스트 사례를 실행하고 통과 여부를 출력한다."""
3     passed = 0
4     failures = []
5
6     for inputs, expected in TEST_CASES:
7         actual = rule(*inputs)
8         if actual == expected:
9             passed += 1
10        else:
11            failures.append((inputs, expected, actual))
12
13    total = len(TEST_CASES)
14    print(" 테스트 결과: {} / {} 통과".format(passed, total))
15
16    for inputs, expected, actual in failures:
17        bits = " ".join(str(int(value)) for value in inputs)
18        print(
19            " 실패 - 입력 ({}): 예상 {}, 실제 {}".format(
20                bits, int(expected), int(actual)
21            )
22        )
23
24    if not failures:
25        print(" 모든 입력 조합에서 의도한 대로 작동한다.")
26    return not failures

```

`run_tests`는 모든 테스트 사례를 순서대로 실행하고, `expected`와 `actual`을 비교하여 통과 개수와 실패한 입력 조합을 보여 준다. 마지막에 “8/8 통과”가 나오면 설계대로 작동한다는 뜻이다.

테스트 문제 3.5 공통 판단 함수 검사

- (1) 제1장 예제 1.2의 우산 채깅 알림 진리표를 꺼낸다.
- (2) `02_truth_table_lab.py`의 `TEST_CASES`가 그 진리표의 예상 출력을 먼저 정확히 옮겼는지 확인한다.
- (3) 그다음 `decide` 함수가 $A = R \wedge O \wedge \neg U$ 를 그대로 구현하는지 확인한다.
- (4) 파일을 실행하여 “8/8 통과”가 나오는지 확인한다.

이 활동은 모든 학생이 같은 파일로 하는 개인 공통 연습이다. 오늘은 `02_truth_table_lab.py`를 개인 또는 팀 프로젝트로 바꾸지 않는다. 팀의 최종 논리는 2일차에 아이디어를 공유하고 팀을 만든 뒤, 선택한 스타터 파일에서 구현한다.

통과하지 못한 행이 있다면 둘 중 하나다. 진리표가 틀렸거나, `decide` 함수가 틀렸다. 어느 쪽인지 판단하려면 그 행의 상황을 말로 읽어 보고 “이때 정말 이 값이 나와야 하는가”를 각자 다시 확인한다.

3.6. 고장 난 판단 함수 수리하기

실습 파일에는 일부러 잘못 만든 판단 함수 두 개가 들어 있다. 학생들이 실제로 가장 많이 하는 두 가지 실수다.

```
1 def decide_broken_1(rain: bool, going_out: bool, has_umbrella: bool) -> bool:
2     """오작동 예 1: and 를 or 로 잘못 쓴 경우."""
3     return rain or going_out or not has_umbrella
4
5
6 def decide_broken_2(rain: bool, going_out: bool, has_umbrella: bool) -> bool:
7     """오작동 예 2: not 을 빠뜨린 경우."""
8     return rain and going_out and has_umbrella
```

같은 TEST_CASES로 이 두 함수를 검사하면 다음과 같이 실패한다.

[3] 오작동 예 1 (and 대신 or)

R O U | A
-----+---

0 0 0 | 1
0 0 1 | 0
0 1 0 | 1
0 1 1 | 1
1 0 0 | 1
1 0 1 | 1
1 1 0 | 1
1 1 1 | 1

테스트 결과: 2 / 8 통과

실패 - 입력 (0 0 0) : 예상 0, 실제 1
실패 - 입력 (0 1 0) : 예상 0, 실제 1
실패 - 입력 (0 1 1) : 예상 0, 실제 1
실패 - 입력 (1 0 0) : 예상 0, 실제 1
실패 - 입력 (1 0 1) : 예상 0, 실제 1
실패 - 입력 (1 1 1) : 예상 0, 실제 1

실습 문제 3.6 오작동 수리 절차

오작동을 고칠 때는 코드를 아무렇게나 바꾸지 않고 다음 순서를 지킨다.

단계 1. 실패한 입력 조합을 확인한다.

단계 2. 그 입력이 뜻하는 상황을 말로 읽는다. 예를 들어 (0 0 0)은 “비도 오지 않고, 외출도 안 하고, 우산도 없다”이다.

단계 3. 자연어 작동 조건과 비교하여 무엇이 옳은지 정한다.

단계 4. 논리식을 확인하고, 필요한 연산자를 고친다.

단계 5. 테스트를 다시 실행하여 모든 행이 통과하는지 본다.

활동. decide_broken_1과 decide_broken_2에 대해 다음 표를 채우시오.

고장 난 함수	통과 개수	무엇을 어떻게 고쳐야 하는가
decide_broken_1		
decide_broken_2		

설계 관점. 테스트 결과를 볼 때 실패를 두 종류로 나누어 보면 원인을 훨씬 빨리 찾는다.

- 쓸데없이 작동 — 예상 0인데 실제 1. 사용자가 귀찮아하고, 결국 장치를 꺼 버린다. 조건이 너무 느슨하다는 신호이므로 AND로 조건을 더 묶어야 한다.
- 필요할 때 침묵 — 예상 1인데 실제 0. 장치가 있어도 문제가 해결되지 않는다. 안전 장치라면 위험하다. 조건이 너무 엄격하다는 신호이므로 불필요한 조건을 빼거나 OR로 경로를 열어야 한다.

`decide_broken_1`은 앞의 잘못이 많고, `decide_broken_2`는 두 잘못을 하나씩 가지고 있다. 어느 쪽이 더 위험한 고장인지 각자 판단하고 이유를 적어 보자.

도전 문제 3.7 진리표에서 논리식 되찾기 [선택]

`decide` 함수의 내용을 지우고, 대신 `TEST_CASES`만 보고 논리식을 다시 만들어 보자. 제1장의 세 단계, 즉 출력이 1인 행을 찾고, 그 행에서만 참이 되는 식을 만들고, V로 연결하는 방법을 그대로 Python으로 옮기면 된다. 출력이 1인 행이 두 개 이상인 진리표로 시도하면 `or`가 필요해진다.

3.7. GUI 맛보기와 개인 아이디어 준비

`code` 폴더의 `03_umbrella_gui.py`를 각자 실행한다. 체크박스를 여러 조합으로 켜고 끄면서 `decide`가 판단하고 화면에 출력이 나타나는지, 전체 테스트에서 “8/8 통과”가 나오는지 확인한다. GUI는 새로운 논리를 만드는 부분이 아니라, 이미 검증한 입력과 출력을 사용자가 조작하고 읽을 수 있게 보여 주는 부분이다.

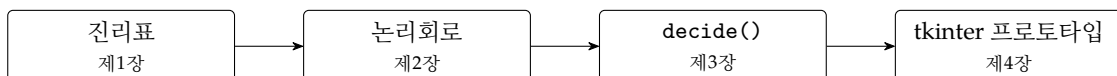
준비 활동 3.8 개인 아이디어 카드

내가 관심 있는 문제를 하나 떠올리고 다음 다섯 줄을 짧게 채운다. 완성된 프로그램이나 개인용 진리표를 만드는 활동은 아니다.

관심 있는 문제 또는 분야	
문제가 생기는 상황	
입력 후보 2 ~ 3개	
원하는 출력	
한 문장 조건	만약 _____ 이고/또는 _____ 이면, _____ 한다.

아이디어가 분명하다면 간단한 논리식이나 일부 경우를 생각해 보아도 좋지만, 개인 진리표 완성은 필수가 아니다. 최종 진리표는 2일차에 팀이 함께 확정한다.

다음 날에는 이 카드를 짧게 공유하고 다른 아이디어를 둘러본 뒤 팀을 만든다. 팀은 한 아이디어를 고르거나 여러 아이디어를 합치거나 새 주제를 정하여 같은 구조의 프로토타입을 만든다.



`decide` 함수는 네 단계를 잇는 다리다. 오늘 공통 예제로 익힌 검증 순서를 2일차 팀 발명품에도 그대로 적용한다.

4. tkinter 프로토타입 제작 Lab

2일차 — 팀 발명 활동. 1일차에 각자 준비한 아이디어를 짧게 공유하고 팀을 만든다. 팀은 문제와 입출력, 판단 논리와 진리표에 합의한 뒤 Python과 tkinter로 사용자가 직접 조작하며 시험할 수 있는 디지털 프로토타입을 완성한다. 마지막에는 모듈끼리 교차 테스트한 뒤 발표한다. A 트랙은 가상 센서와 출력 장치를, B 트랙은 비트 입력과 Sum/Cout 표시를 붙인다. 붙이는 대상만 다를 뿐 방법은 같다.

차시	시간	활동
1차시	0 ~ 10분	개인 아이디어 발표와 다른 아이디어 둘러보기
	10 ~ 20분	2 ~ 4명 팀 구성, 팀 아이디어 선택·결합
	20 ~ 50분	문제, 입출력, 자연어 조건, 논리식, 최종 진리표 합의
2차시	0 ~ 15분	tkinter 구조 이해, 1일차 우산 GUI 다시 보기
	15 ~ 25분	최종 진리표에서 TEST_CASES 먼저 작성
	25 ~ 50분	decide() 구현, GUI 팀 프로토타입 제작
3차시	0 ~ 20분	프로토타입 완성, 모든 입력 조합 전체 테스트
	20 ~ 35분	오작동 수정과 사용자 인터페이스 개선
	35 ~ 50분	모듈 간 교차 테스트, 1분 발표

4.1. 개인 아이디어 공유와 팀 구성

먼저 한 사람당 약 30초 동안 개인 아이디어 카드에서 딱 세 가지만 말한다.

- (1) 해결하고 싶은 문제
- (2) 사용할 수 있는 입력
- (3) 원하는 출력

예를 들어 “혼자 있는 반려동물이 너무 더워지는 문제를 해결하고 싶다. 입력은 온도가 높음과 사람이 집에 있음이고, 출력은 환기 알림이다”처럼 말하면 된다. 발표가 끝나면 약 5분 동안 다른 사람의 아이디어를 둘러본다. 비슷한 문제를 고른 사람, 아이디어를 합칠 수 있는 사람, 또는 함께 만들어 보고 싶은 서로 다른 관점을 가진 사람이 2 ~ 4명씩 팀을 이룬다. 팀은 한 아이디어를 발전시키거나 여러 아이디어를 합치거나 새 아이디어를 정할 수 있다.

코드를 열기 전에 팀이 해결할 문제, 입력, 출력, 자연어 작동 조건, 논리식, 최종 진리표를 차례로 합의한다. 그 뒤에는 반드시 다음 순서로 작업한다.

문제 → 입출력 → 자연어 조건 → 논리식 → 진리표 → TEST_CASES → decide() → GUI

진리표는 팀 제품이 어떻게 작동해야 하는지를 정한 사양이므로, 그 사양을 TEST_CASES로 먼저 옮기고 나서 decide()를 구현한다. 화면 제작은 나누어 맡아도 되지만, 모든 팀원이 논리식, 진리표, 테스트 사례와 decide()가 어떻게 이어지는지 설명할 수 있어야 한다.

4.2. 왜 tkinter로 프로토타입을 만드는가

tkinter는 별도로 설치하는 응용 프로그램이 아니라 Python 표준 GUI 라이브러리다([Python Software Foundation, 2024b](#)). 따로 설치할 것이 없는 경우가 대부분이지만, 배포판이나 설치 방식에 따라 tkinter가 빠져 있어 사용할 수 없는 환경도 있으므로 수업 전에 반드시 확인한다.

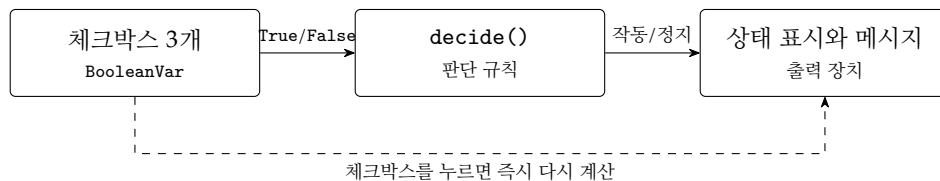
tkinter를 쓰면 우리가 쓴 판단 규칙과 사용자가 보는 화면이 분리된다. 이 분리는 실제 스마트 장치의 구조와 똑같다. GUI는 새로운 논리를 만드는 것이 아니라, 이미 정한 입력을 조작하고 출력을 알아보기 쉽게 표시하는 역할을 한다.

실제 스마트 장치	tkinter 프로그램	설명
센서	<code>ttk.Checkbutton</code> , <code>BooleanVar</code>	참/거짓 상태를 사용자가 켜고 끈다
판단 회로	<code>decide()</code> 함수	1일차 공통 예제로 익히고, 팀 진리표 뒤에 구현하는 부분
경고등, 부저, 환풍기	<code>Label</code> , 상태 표시, 메시지	출력이 작동하는지 눈으로 보인다
센서 상태 변화	체크박스 클릭과 <code>command</code> 함수	상태가 바뀔 때마다 판단을 다시 한다

무엇보다 중요한 차이는 이것이다. 계산 결과를 한 번 출력하고 끝나는 프로그램과 달리, `tkinter` 프로그램은 실행된 상태로 남아 사용자의 조작을 계속 기다린다. 그래서 사용자가 센서를 이리저리 바꾸며 발명품을 시험할 수 있다. 이것이 프로토타입이다.

4.3. 입력-판단-출력 구조

모든 실습 파일은 다음 구조를 공유한다.



체크박스를 누를 때마다 판단 함수가 다시 실행되고 출력이 갱신된다. “실행” 버튼을 따로 누르지 않아도 되는 이런 구조를 이벤트 기반 구조라 한다. 실제 스마트 장치도 센서 값이 바뀔 때마다 스스로 판단을 다시 한다.

4.4. tkinter의 최소 개념

`tkinter`의 모든 기능을 알 필요는 없다. 이번 수업에 필요한 것은 일곱 가지뿐이다.

문법	역할
<code>tk.Tk()</code>	프로그램 창을 만든다
<code>tk.BooleanVar()</code>	센서 하나의 참/거짓 상태를 담아 둔다
<code>ttk.Checkbutton</code>	가상 센서. 사용자가 켜고 끈다
<code>ttk.Label</code> , <code>tk.Label</code>	출력 상태와 메시지를 보여 준다
<code>ttk.Button</code>	초기화, 전체 테스트, 종료 버튼
<code>command=</code> 함수	상태가 바뀌거나 버튼을 누를 때 실행할 함수
<code>mainloop()</code>	프로그램이 계속 사용자 입력을 기다리게 한다

`ttk`는 `tkinter`의 위젯을 조금 더 현대적인 모양으로 그려 주는 하위 모듈이다. 기본적으로 `ttk`를 쓰고, 배경색을 직접 지정해야 하는 상태 표시에만 `tk.Label`을 쓴다.

주의. 창을 만들고 위젯을 배치하는 코드는 이미 완성되어 있다. 학생은 판단 규칙과 화면에 나올 문구만 고친다. 클래스, 상속, 복잡한 이벤트 처리는 이번 수업의 학습 목표가 아니다.

위젯 이름을 외울 필요도 없다. 필요할 때 이 표를 보면 된다.

4.5. 코드의 두 영역

모든 템플릿 파일은 두 영역으로 나뉘어 있고, 주석으로 명확히 구분되어 있다.

영역	내용
# ===== # 학생이 수정하는 부분 # =====	TITLE, DESCRIPTION, INPUT_LABELS, OUTPUT_LABEL, MESSAGE_ON, MESSAGE_OFF, TEST_CASES, decide()
# ===== # GUI 프레임워크: # 처음에는 수정하지 않음 # =====	창 생성, 체크박스 생성, 버튼 생성, 배치, 결과 갱신, 테스트 실행, mainloop

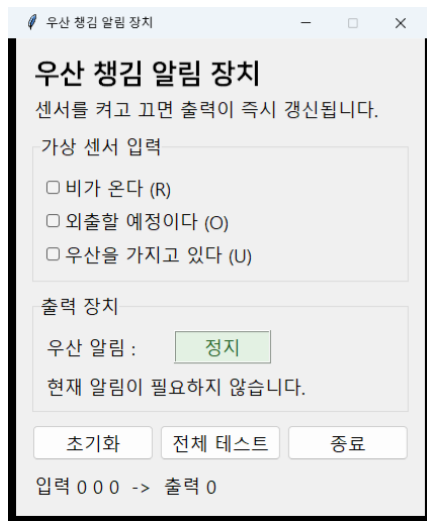
주의. INPUT_LABELS의 개수, decide()의 매개변수 개수, TEST_CASES의 입력 개수는 반드시 같아야 한다. 하나만 달라도 다음과 같은 오류가 난다.

```
TypeError: decide() takes 3 positional arguments but 4 were given
```

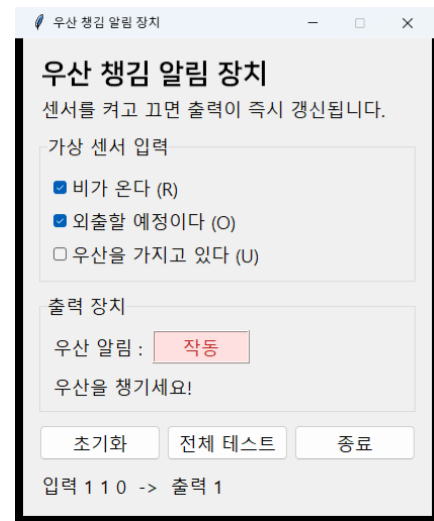
이 오류가 나면 세 곳의 개수를 세어 본다.

4.6. Lab 1. 공통 실습: 우산 챙김 알림 장치

1일차에 모든 학생이 각자 실행했던 같은 프로그램을 다시 연다. code 폴더의 03_umbrella_gui.py를 실행하면 그림 4.1과 같은 창이 열린다.



(a) 모든 센서가 꺼진 상태. 알림이 정지되어 있다.



(b) $R = 1, O = 1, U = 0$. 알림이 작동한다.

그림 4.1: 우산 챙김 알림 장치의 실행 화면. 아래쪽에 현재 입력과 출력이 0과 1로 표시되어 진리표와 바로 비교할 수 있다.

제품 시험표는 진리표에서 먼저 그대로 옮긴 것이다.

```
1 # 제품 시험표: 종이에 쓴 진리표의 예상 출력을 그대로 옮겨 적는다.
2 # ((비, 외출, 우산), 예상 출력)
3 TEST_CASES = [
4     ((False, False, False), False),
5     ((False, False, True), False),
```



```

6      ((False, True, False), False),
7      ((False, True, True), False),
8      ((True, False, False), False),
9      ((True, False, True), False),
10     ((True, True, False), True),
11     ((True, True, True), False),
12 ]

```

그 사양을 검사할 판단 함수는 제3장에서 쓴 것과 완전히 같다.

```

1 def decide(rain: bool, going_out: bool, has_umbrella: bool) -> bool:
2     """판단 규칙: A = R and O and (not U)"""
3     # 비가 오고, 외출할 예정이며, 우산이 없을 때에만 알린다.
4     return rain and going_out and not has_umbrella

```

제목, 입출력 문구와 판단 함수가 들어 있는 학생 수정 영역은 다음과 같다.

```

1 # 학생이 수정하는 부분
2 # =====
3
4 TITLE = "우산 챙김 알림 장치"
5 DESCRIPTION = "센서를 켜고 끄면 출력이 즉시 갱신됩니다."
6
7 # 가상 센서(입력)의 이름
8 INPUT_LABELS = [
9     "비가 온다 (R)",
10    "외출할 예정이다 (O)",
11    "우산을 가지고 있다 (U)",
12 ]
13
14 # 출력 장치의 이름과 상태 문구
15 OUTPUT_LABEL = "우산 알림"
16 MESSAGE_ON = "우산을 챙기세요!"
17 MESSAGE_OFF = "현재 알림이 필요하지 않습니다."

```

센서와 판단 함수를 잇는 부분

GUI 프레임워크에서 핵심은 다음 함수 하나다. 체크박스 상태를 모아 decide에 넣고, 그 결과로 화면을 바꾼다.

```

1 def update_output(*args) -> None:
2     """센서 상태가 바뀔 때마다 판단 함수를 다시 실행한다."""
3     values = [var.get() for var in sensor_vars]
4     result = decide(*values)
5     state_label.config(
6         text=TEXT_ON if result else TEXT_OFF,
7         fg=FG_ON if result else FG_OFF,
8         bg=BG_ON if result else BG_OFF,
9     )
10    message_label.config(text=MESSAGE_ON if result else MESSAGE_OFF)
11    status_label.config(
12        text="입력 {} -> 출력 {}".format(bits(values), int(result))
13    )

```

그리고 각 체크박스에 command=update_output을 붙여 두면, 사용자가 체크박스를 누를 때마다 이 함수가 자동으로 실행된다.

```

1 for row, (label, var) in enumerate(zip(INPUT_LABELS, sensor_vars)):
2     ttk.Checkbutton(
3         sensor_box, text=label, variable=var, command=update_output

```

```
4         ).grid(row=row, column=0, sticky="w", pady=1)
```

버튼 세 개도 같은 방식으로 함수를 연결한다.

```
1     def reset_sensors() -> None:
2         for var in sensor_vars:
3             var.set(False)
4             update_output()
5
6     def show_test_report() -> None:
7         messagebox.showinfo("전체 테스트", test_report())
8
9     ttk.Button(button_box, text="초기화", command=reset_sensors).pack(
10         side="left"
11     )
12     ttk.Button(button_box, text="전체 테스트", command=show_test_report).pack(
13         side="left", padx=6
14     )
15     ttk.Button(button_box, text="종료", command=root.destroy).pack(side="right")
```

실습 문제 4.1 공통 실습 프로그램 살펴보기

- (1) 03_umbrella_gui.py를 실행하고 체크박스 세 개를 여러 조합으로 켜 보며, 출력이 작동하는 조합이 정확히 하나뿐임을 확인한다.
- (2) 창 아래쪽의 “입력 1 1 0 → 출력 1” 표시를 보면서 제1장 진리표의 해당 행을 찾는다.
- (3) 전체 테스트 버튼을 눌러 “8/8 통과”를 확인한다.
- (4) decide 함수의 not has_umbrella를 has_umbrella로 바꾸고 저장한 뒤 다시 실행한다. 전체 테스트가 몇 개 통과하는가? 어느 조합이 실패하는가?
- (5) MESSAGE_ON을 “우산!”로 바꾸어 보고, 원래 문구와 비교해 어느 것이 사용자에게 더 도움이 되는지 이야기한다.
- (6) 고친 부분을 모두 원래대로 되돌린다.

4.7. Lab 2 (A 트랙). 팀별 스마트 발명품

여기서부터 트랙이 나뉜다. A 트랙은 이 절을, B 트랙은 다음 절을 따라가면 된다. 두 절의 작업 순서는 완전히 같으므로, 먼저 끝낸 모듈은 다른 트랙의 절을 읽어 보아도 좋다.

이제 우리 팀 발명품을 만든다. 템플릿을 팀 폴더에 복사한 뒤 수정한다.

난이도	사용 파일	조건
기본	04_invention_starter_1out.py	입력 2개, 출력 1개, and 또는 or
표준	04_invention_starter_1out.py	입력 3개, 출력 1개, not 포함, 8행 진리표
도전	05_invention_starter_2out.py	입력 3개, 출력 2개, 출력마다 다른 조건식, UI 개선

완성 기준은 표준 수준이다. 기본 수준으로 먼저 작동시킨 뒤 표준으로 올리는 것도 좋은 방법이다. 표준을 완성한 팀은 도전 수준으로 확장한다. NAND만으로 만든 회로는 수학 심화 과제이며 프로토타입의 필수 조건이 아니다. (B 트랙은 07_full_adder_starter.py 하나를 사용하며, 난이도는 A 트랙의 도전 수준에 해당한다.)

템플릿에서 고칠 부분

템플릿의 학생 수정 영역은 처음부터 예시 발명품(스마트 복도 절전 조명)으로 채워져 있다. 따라서 아무것도 고치지 않아도 프로그램은 그대로 실행된다. 그러나 예시를 팀 제품의 사양으로 사용하지 않는다. 먼저 최종 진리표의 예상 출력을 제품 시험표에 옮긴다.

```
1 # 제품 시험표: 종이에 쓴 진리표의 예상 출력을 그대로 옮겨 적는다.
2 # ((입력1, 입력2, 입력3), 예상 출력)
3 TEST_CASES = [
4     ((False, False, False), False),
5     ((False, False, True), False),
6     ((False, True, False), False),
7     ((False, True, True), False),
8     ((True, False, False), False),
9     ((True, False, True), False),
10    ((True, True, False), True),
11    ((True, True, True), False),
12 ]
```

그다음 아래 노란 영역을 우리 팀 발명품에 맞게 바꾼다.

```
1 # 학생이 수정하는 부분
2 # =====
3
4 TITLE = "스마트 복도 절전 조명"
5 DESCRIPTION = "센서를 켜고 끄면 출력이 즉시 갱신됩니다."
6
7 # 가상 센서(입력)의 이름 - 사용자가 뜻을 바로 알 수 있게 쓴다.
8 INPUT_LABELS = [
9     "사람이 있다 (P)",
10    "주변이 어둡다 (D)",
11    "수동 소등 스위치가 눌렀다 (M)",
12 ]
13
14 # 출력 장치의 이름과 상태 문구
15 OUTPUT_LABEL = "복도 조명"
16 MESSAGE_ON = "조명을 켭니다."
17 MESSAGE_OFF = "조명을 끈 상태로 전기를 절약합니다."
18
19
20 def decide(person: bool, dark: bool, manual_off: bool) -> bool:
21     """판단 규칙: L = P and D and (not M)"""
22     # 매개변수의 개수와 순서는 INPUT_LABELS와 똑같이 맞춘다.
23     return person and dark and not manual_off
```

프로젝트 카드

주제를 정하지 못한 팀은 아래에서 고른다. 논리식은 일부만 예시로 주어져 있으므로, 나머지는 팀이 직접 설계한다.

발명품	입력 후보	출력 후보 / 예시 조건	난이도
스마트 복도 절전 조명	사람 있음, 어두움, 수동 소등	조명. $L = P \wedge D \wedge \neg M$	표준
우산 챙김 알림 장치	비, 외출 예정, 우산 소지	알림. $A = R \wedge O \wedge \neg U$	표준
스마트 과학실 안전관리	가스, 고온, 사람 있음	대피 정보와 환풍기. 조건식을 다르게 설계	도전
스마트 사물함 출입 제어	카드 인식, 사용 가능 시간, 긴급 승인	잠금 해제. 우회 조건을 어떻게 다룰지 결정	표준
냉장고 문 열림 경고	문 열림, 일정 시간 경과, 야간	부저와 표시등. 야간에는 소리를 줄일지 결정	도전
야간 보행 안전등	어두움, 보행자 접근, 배터리 부족	안전등과 배터리 경고	도전
반려식물 물주기 알림	흙이 건조함, 마지막 급수 후 시간 경과, 사람이 집에 있음	알림. 사람이 없을 때 알릴지 결정	표준
교실 환기 알림	이산화탄소 농도 높음, 수업 중, 창문 닫힘	환기 알림. 수업 중 알릴지 결정	표준

실습 문제 4.2 팀 프로토타입 만들기

단계 1. 설계 문제 2.4에서 팀의 문제, 입출력, 자연어 조건, 논리식과 최종 진리표를 확정한다.

단계 2. 템플릿을 팀 폴더에 복사하고 파일 이름을 `team03_invention.py` 처럼 바꾼다. 원본은 그대로 둔다.

단계 3. `TEST_CASES`의 예상 출력을 최종 진리표에서 먼저 옮겨 적는다.

단계 4. `decide` 함수를 우리 팀 논리식으로 바꾼다. 매개변수 순서를 팀이 정한 입력 순서와 맞춘다.

단계 5. `TITLE`과 `DESCRIPTION`을 우리 발명품에 맞게 바꾼다.

단계 6. `INPUT_LABELS`를 우리 팀 입력 이름으로 바꾼다. 사용자가 설명 없이 뜻을 알 수 있게 쓴다.

단계 7. `OUTPUT_LABEL`과 두 상태 문구를 정한다.

단계 8. 실행하여 창이 열리는지 확인하고, 전체 테스트를 눌러 결과를 본다.

템플릿의 처음 상태에도 예시 발명품이 들어 있으므로, 아직 아무것도 고치지 않은 상태에서도 프로그램은 정상적으로 실행된다. 팀의 진리표와 `TEST_CASES`, `decide`를 정한 뒤 실행하여 검사한다.

출력이 두 개인 경우

도전 수준에서는 판단 함수가 두 값을 함께 돌려준다. 야간 보행 안전등 템플릿의 판단 함수는 다음과 같다.

```

1 def decide(dark: bool, walking: bool, battery_low: bool):
2     """판단 규칙: 두 출력의 목적이 다르므로 조건식도 다르다."""
3     # 안전등은 사람이 지나갈 때만 켜서 전력을 절약한다.
4     # 배터리 경고는 사람이 없어도 관리자에게 알려야 한다.
5     light = dark and walking and not battery_low
6     battery_warning = battery_low
7     return light, battery_warning

```

안전등은 사람이 지나갈 때만 켜서 전력을 아끼지만, 배터리 경고는 사람이 없어도 관리자에게 알려야 한다. 그래서 두 조건식이 다르다. 출력 이름과 문구도 두 개씩 짝을 맞추어 적는다.

```

1 # 출력 장치 두 개의 이름과 상태 문구 (순서를 맞춘다)
2 OUTPUT_LABELS = ["보행 안전등", "배터리 경고"]
3 MESSAGES_ON = [

```

```

4     "보행자를 위해 안전등을 켭니다.",
5     "배터리를 교체하세요.",
6 ]
7 MESSAGES_OFF = [
8     "안전등을 끈 상태로 전력을 절약합니다.",
9     "배터리 상태는 정상입니다.",
10 ]

```

4.8. Lab 2 (B 트랙). 이진 계산 장치

B 트랙은 1일차에 유도한 전가산기를 그대로 프로토타입으로 만든다. 아이디어 공유 뒤 이 트랙을 선택한 팀은 만들 장치가 이미 정해져 있으므로, 대신 “정말 덧셈이 맞는가”를 두 가지 방법으로 검증하는 데 시간을 쓴다.

사용 파일은 07_full_adder_starter.py이며, 구조는 A 트랙 템플릿과 완전히 같다. 체크박스가 센서 대신 비트 A , B , C_{in} 을 나타내고, 출력 두 개가 Sum과 Cout일 뿐이다.

제품 시험표

연습문제 2.3에서 채운 진리표를 먼저 그대로 옮겨 적는다.

```

1 # 제품 시험표: ((A, B, Cin), (예상 Sum, 예상 Cout))
2 TEST_CASES = [
3     ((False, False, False), (False, False)),
4     ((False, False, True), (True, False)),
5     ((False, True, False), (True, False)),
6     ((False, True, True), (False, True)),
7     ((True, False, False), (True, False)),
8     ((True, False, True), (False, True)),
9     ((True, True, False), (False, True)),
10    ((True, True, True), (True, True)),
11 ]

```

XOR를 직접 만든다

Python에는 XOR에 해당하는 $\sim$ 연산자가 있지만 쓰지 않는다. 제1장 도전 문제 1.2에서 유도한 대로 $P \oplus Q \equiv (P \wedge \neg Q) \vee (\neg P \wedge Q)$ 를 and, or, not으로 옮겨 직접 만든다. 그래야 우리가 세운 논리식이 그대로 코드가 된다.

```

1 # 학생이 수정하는 부분
2 # =====
3
4 TITLE = "한 자리 이진 덧셈기"
5 DESCRIPTION = "비트를 켜고 끄면 합과 올림이 즉시 갱신됩니다."
6
7 INPUT_LABELS = [
8     "A = 1",
9     "B = 1",
10    "Cin = 1 (아래 자리에서 올라온 올림)",
11 ]
12
13 OUTPUT_LABELS = ["Sum (합의 자리)", "Cout (올림)"]
14 MESSAGES_ON = [
15     "합의 자리가 1입니다.",
16     "윗자리로 1을 올립니다.",
17 ]
18 MESSAGES_OFF = [
19     "합의 자리가 0입니다.",

```

```

20     "윗자리로 올릴 것이 없습니다.",
21 ]
22
23
24 def xor(a: bool, b: bool) -> bool:
25     """XOR: 두 입력이 서로 다를 때만 참"""
26     return (a and not b) or (not a and b)
27
28
29 def decide(a: bool, b: bool, carry_in: bool):
30     """전가산기: 반가산기 두 개와 OR 하나를 이어 붙인 구조"""
31     s1 = xor(a, b)
32     digit = xor(s1, carry_in)
33     carry_out = (a and b) or (s1 and carry_in)
34     return digit, carry_out

```

decide 함수를 1일차의 유도과 나란히 놓고 보자.

$S_1 = A \oplus B$	<code>s1 = xor(a, b)</code>
$\text{Sum} = S_1 \oplus C_{in}$	<code>digit = xor(s1, carry_in)</code>
$\text{Cout} = (A \wedge B) \vee (S_1 \wedge C_{in})$	<code>carry_out = (a and b) or (s1 and carry_in)</code>

종이에 쓴 식과 코드가 한 줄씩 정확히 대응한다. 반가산기 두 개를 이어 붙였다는 구조도 코드에 그대로 남아 있다.

두 가지 방법으로 검증한다

B 트랙의 장점은 정답을 확인하는 방법이 두 가지라는 점이다.

- (1) 진리표 비교. TEST_CASES의 예상 출력과 실제 출력을 비교한다. A 트랙과 같은 방법이다.
- (2) 덧셈 계산. 여덟 가지 조합 모두에서 $A + B + C_{in} = \text{Sum} + 2\text{Cout}$ 이 성립하는지 확인한다. 이 검사는 진리표를 보지 않고도 통과 여부를 알려 준다.

두 번째 검사는 다음과 같이 구현되어 있다.

```

1 def run_arithmetic_check():
2     """덧셈 계산. A + B + Cin 이 Sum + 2*Cout 과 같은지 확인한다."""
3     passed = 0
4     failures = []
5     for values in all_input_cases():
6         digit, carry_out = outputs_of(values)
7         left = sum(int(value) for value in values)
8         right = int(digit) + 2 * int(carry_out)
9         if left == right:
10             passed += 1
11         else:
12             failures.append(
13                 "입력 ({}): {} != {} + 2*{}".format(
14                     bits(values), left, int(digit), int(carry_out)
15                 )
16             )
17     return passed, len(all_input_cases()), failures

```

진리표를 잘못 옮겨 적으면 (1)은 실패하지만 (2)는 통과한다. 반대로 논리식이 틀리면 두 검사가 함께 실패한다. 어느 쪽이 실패했는지를 보면 무엇이 틀렸는지 바로 알 수 있다.

실습 문제 4.3 전가산기 프로토타입 만들기

단계 1. 07_full_adder_starter.py를 팀 폴더에 복사하고 team05_adder.py처럼 이름을 바꾼다.

단계 2. TEST_CASES를 설계 문제 2.5의 최종 진리표에서 먼저 옮겨 적는다.

단계 3. xor 함수를 지우고 직접 다시 써 본다. and, or, not만 사용한다. 이어서 decide가 전가산기 논리식과 일치하는지 확인한다.

단계 4. 실행하여 GUI 창이 열리는지 확인한다.

단계 5. 전체 테스트를 눌러 두 검사가 모두 8/8인지 확인한다.

단계 6. 체크박스를 눌러 가며 화면의 식 $1 + 1 + 1 = 11_{(2)} = 3$ 이 맞는지 눈으로 확인한다.

실행하면 터미널에 다음과 같이 진리표와 두 검사 결과가 함께 출력된다.

```
=====
한 자리 이진 덧셈기
=====
  A  B  Cin | Sum  Cout | A+B+Cin  Sum+2*Cout
-----+-----+-----
  0  0   0 |   0   0 |       0       0
  0  0   1 |   1   0 |       1       1
  0  1   0 |   1   0 |       1       1
  0  1   1 |   0   1 |       2       2
  1  0   0 |   1   0 |       1       1
  1  0   1 |   0   1 |       2       2
  1  1   0 |   0   1 |       2       2
  1  1   1 |   1   1 |       3       3

[1] 진리표 비교: 8 / 8 통과

[2] 덧셈 검사 A + B + Cin = Sum + 2*Cout : 8 / 8 통과

여덟 가지 조합 모두에서 올바르게 덧셈을 수행합니다.
=====
```

도전 문제 4.4 두 자리 이진 덧셈기 [선택]

전가산기 하나는 한 자리를 더한다. 두 자리 이진수 $(A_1A_0)_{(2)}$ 와 $(B_1B_0)_{(2)}$ 를 더하려면 전가산기를 두 개 이어 붙이면 된다. 핵심은 오른쪽 자리의 Cout을 왼쪽 자리의 Cin으로 연결하는 것이다.

decide 함수를 두 번 호출하기만 하면 된다.

```
1 def add_two_bits(a1: bool, a0: bool, b1: bool, b0: bool):
2     """두 자리 이진수 (a1 a0) 와 (b1 b0) 를 더한다."""
3     digit0, carry0 = decide(a0, b0, False)
4     digit1, carry1 = decide(a1, b1, carry0)
5     return carry1, digit1, digit0
```

(1) 가장 오른쪽 자리의 Cin에는 왜 False를 넣는가?

(2) 결과가 왜 세 자리 (carry1, digit1, digit0)가 되는가? 두 자리 수 두 개를 더하면 가장 클 때 얼마인가?

(3) 16가지 입력 조합을 모두 시험하여 $(A_1A_0)_{(2)} + (B_1B_0)_{(2)}$ 가 십진수로도 맞는지 확인하시오.

(4) 세 자리, 네 자리로 늘리려면 무엇을 반복하면 되는지 설명하시오.

완성된 예시는 code/solutions/full_adder_solution.py에 있다. 네 자리 이상으로 늘리는 것은 이번 수업의 목표가 아니다. “자리를 늘리는 방법을 안다”는 것으로 충분하다.

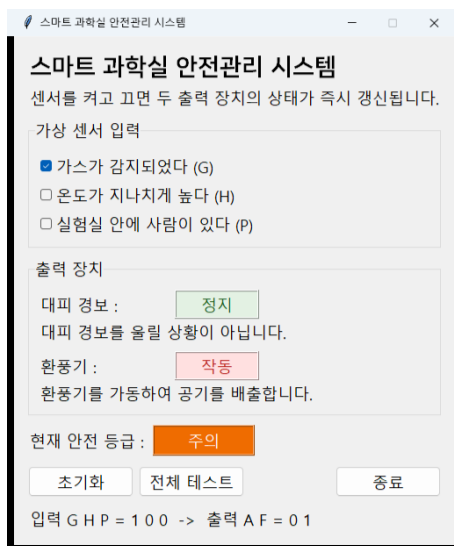
4.9. 완성 예시: 스마트 과학실 안전관리 시스템

제2장에서 설계한 안전관리 시스템은 06_safety_system_example.py에 완성되어 있다. 두 출력의 목적이 다른 조건식도 달라야 한다는 원리를 화면에서 직접 확인할 수 있다.

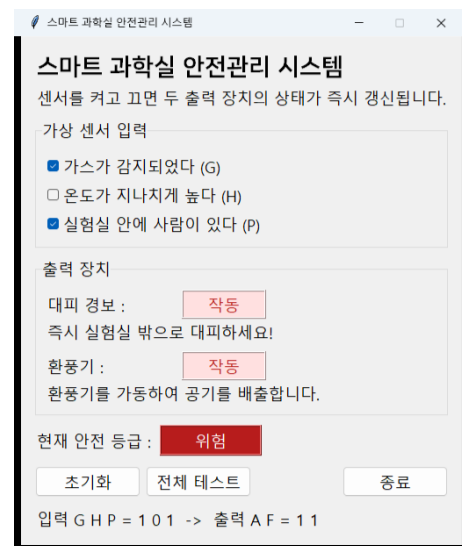
```
1 def decide(gas: bool, high_temp: bool, person: bool):
2     """두 출력의 목적이 다르므로 조건식도 다르다."""
3     danger = gas or high_temp
4     alarm = danger and person
5     fan = danger
6     return alarm, fan
```

이 프로그램은 사용자가 판단 근거를 이해할 수 있도록 안전 등급도 함께 보여 준다.

```
1 def safety_level(alarm: bool, fan: bool):
2     """사용자에게 보여 줄 상태 등급을 정한다."""
3     if alarm:
4         return LEVEL_DANGER
5     if fan:
6         return LEVEL_CAUTION
7     return LEVEL_NORMAL
```



(a) $G = 1, P = 0$. 사람이 없으므로 경보는 정지하지만 환풍기는 작동한다. 등급은 “주의”.



(b) $G = 1, P = 1$. 사람이 있으므로 경보와 환풍기가 함께 작동한다. 등급은 “위험”.

그림 4.2: 같은 가스 감지 상황에서 사람의 유무만 달라진 두 화면. 출력의 목적에 따라 작동 여부가 달라진다.

이 프로그램은 실행할 때 터미널에 8가지 입력 조합의 진리표와 자동 테스트 결과를 함께 출력한다.

```
=====
스마트 과학실 안전관리 시스템
=====
```


G H P | A F

```

-----+-----
0 0 0 | 0 0
0 0 1 | 0 0
0 1 0 | 0 1
0 1 1 | 1 1
1 0 0 | 0 1
1 0 1 | 1 1
1 1 0 | 0 1
1 1 1 | 1 1

```

테스트 결과: 8 / 8 통과
출력 순서: 대피 경보, 환풍기

8가지 입력 조합 모두 설계한 대로 작동합니다.

예제 4.1 두 출력을 비교하기

위 진리표에서 다음을 확인하시오.

- (1) A 가 1인 행과 F 가 1인 행의 개수를 각각 세시오.
- (2) $A = 0$ 이고 $F = 1$ 인 행을 모두 찾고, 각각 어떤 상황인지 말로 쓰시오.
- (3) 환풍기 조건을 $F = (G \vee H) \wedge P$ 로 바꾸면 어느 행이 달라지는가? 그 변경이 만드는 위험을 쓰시오.
- (4) 반대로 경보 조건을 $A = G \vee H$ 로 바꾸면 어떤 불편이 생기는가?

4.10. 전체 입력 테스트

입력이 n 개인 프로젝트는 2^n 가지 조합을 모두 시험한다. 입력이 세 개면 8/8, 두 개면 4/4가 통과해야 한다. 템플릿의 전체 테스트 버튼은 TEST_CASES를 실행하고 통과 개수, 전체 개수, 실패한 조합의 예상 출력과 실제 출력을 보여 준다. 진리표 보기 버튼은 모든 조합을 표로 한 번에 보여 준다.

전체 입력 테스트					
입력 1	입력 2	입력 3	예상	실제	일치
0	0	0	0	0	○
0	0	1	0	0	○
0	1	0	0	0	○
0	1	1	0	0	○
1	0	0	0	0	○
1	0	1	0	0	○
1	1	0	1	1	○
1	1	1	0	0	○

예상 열이 '-'이면 그 조합이 TEST_CASES에 없다는 뜻이다.

닫기

그림 4.3: 진리표 보기 창. 예상 열과 실제 열을 나란히 보여 주고, 일치 여부를 표시한다. 예상 열이 “-”이면 그 조합이 TEST_CASES에 빠져 있다는 뜻이다.

TEST_CASES를 아직 다 채우지 않았거나 형식이 잘못되어도 프로그램은 오류로 멈추지 않고, 몇 번째 줄을 어떻게 고쳐야 하는지 알려 준다.

테스트 문제 4.5 전체 입력 조합 기록표

두 트랙 모두 이 표를 작성한다. 자동 테스트로 모든 조합을 확인하고, 손 기록은 실패하거나 예상과 실재가 달랐던 행, 논리식을 고친 뒤 다시 확인한 행과 수정 내용을 중심으로 남긴다. 아래 표를 채운 뒤, 전체 테스트 버튼의 결과와 비교한다. 입력이 두 개인 팀은 위의 네 행만 사용한다. B 트랙은 입력 세 칸이 A, B, C_{in} 이고 출력 칸에 Sum과 Cout을 나란히 적는다.

입력 1	입력 2	입력 3	예상 출력	실제 출력	일치	발견한 문제
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

통과 결과: _____ / _____ 수정한 내용: _____

4.11. 사용자 인터페이스 개선

UI 개선은 화면을 예쁘게 꾸미는 활동이 아니다. 사용자가 장치를 올바르게 이해하고 사용할 수 있게 만드는 설계 작업이다. 다음 질문으로 우리 프로토타입을 점검한다.

점검 질문

예 / 아니오

- 입력의 의미를 설명 없이 이해할 수 있는가?
- 현재 어떤 입력이 켜져 있는지 한눈에 보이는가?
- 출력의 의미를 한눈에 이해할 수 있는가?
- 현재 입력과 출력의 관계를 사용자가 확인할 수 있는가?
- (A) 정상 상태와 위험 상태가 색이나 문구로 구분되는가?
- (A) 경고 문구가 사용자가 해야 할 행동을 알려 주는가?
- (A) 불필요하게 경고가 자주 나타나지는 않는가?
- (B) Sum과 Cout을 이진 덧셈의 결과로 쉽게 읽을 수 있는가?
- (B) 계산 결과를 십진수와도 비교할 수 있는가?
- 처음 상태로 되돌려 다시 시험하기 쉬운가?

논리식이 맞아도 사용자가 이해할 수 없으면 좋은 설계가 아니다.

실습 문제 4.6 개선 항목 고르기

아래에서 최소 하나를 골라 실제로 고친다. 고친 내용을 발표에서 설명한다.

- (1) 입력 조건 이름을 더 명확한 표현으로 바꾼다.
- (2) 위험 상태의 출력 문구를 행동 중심으로 바꾼다. (“위험!” → “창문을 열고 밖으로 나가세요”)
- (3) 정상, 주의, 위험 세 단계를 구분해서 보여 준다. 06_safety_system_example.py의 safety_level 함수를 참고한다.
- (4) 진리표를 다시 검토하여 불필요하게 작동하는 조건을 없앤다.
- (5) 반대로 작동해야 하는데 빠진 조건을 찾아 추가한다.
- (6) 출력 장치를 두 개로 늘리고 각각 다른 조건식을 준다.
- (7) 초기화 버튼의 동작을 개선하거나 안내 문구를 덧붙인다.
- (8) 테스트 결과를 읽기 쉽게 정리한다.

색과 글꼴만 바꾼 것은 개선으로 인정하지 않는다. 사용자의 판단이나 행동이 달라져야 개선이다.

4.12. 모둠 간 교차 테스트

모든 논리 테스트를 통과한 뒤에 교차 테스트를 시작한다. 다른 모듬의 자리로 가서 그 팀의 프로토타입을 사용자로서 사용해 본다. 만든 사람의 설명을 듣지 않고 먼저 혼자 조작해 보는 것이 중요하다. 판단 결과가 진리표와 일치하는지와, 별도 설명 없이도 입력과 출력을 이해할 수 있는지를 함께 확인한다.

테스트 문제 4.7 교차 테스트 평가표

평가 항목	예	아니오
설명을 듣지 않고도 입력의 뜻을 이해할 수 있었다		
체크박스를 누르면 출력이 즉시 갱신되었다		
진리표와 실제 출력이 일치했다		
서로 다른 입력 조합에서 출력 변화가 정확했다		
(A) 필요한 상황에서 작동하고 불필요한 상황에서는 정지했다		
(B) Sum과 Cout이 이진 덧셈 결과와 일치했다		
출력의 의미를 설명 없이 이해할 수 있었다		

가장 좋았던 점: _____

개선 제안: _____

발견한 오작동 조합: _____

교차 테스트에서 받은 지적은 곧바로 고칠 필요는 없다. 다만 발표에서 “이런 지적을 받았고, 이렇게 고칠 계획이다”라고 말할 수 있어야 한다. 실제 제품 개발도 이렇게 진행된다.

4.13. 1분 발표

각 모둠은 프로토타입을 실행한 상태로 1분 안에 발표한다. 아래 틀의 빈칸을 채워 그대로 읽으면 1분이 맞는다.

설계 문제 4.8 1분 발표 대본

1. 우리 팀이 만든 장치의 목표는 _____이다.
2. 이 장치를 쓰는 사람 또는 사용 상황은 _____이다.
3. 입력은 _____, _____, _____이다.
4. 출력은 _____이다.
5. 핵심 논리식은 _____이다.
6. 테스트에서 발견한 문제는 _____이다.
7. 이를 _____ 방식으로 개선했다.
8. 우리 설계의 장점은 _____이다.

시연. 5번의 논리식을 말할 때 실제로 체크박스를 눌러 보인다. A 트랙은 출력이 작동하는 조합과 작동하지 않는 조합을 하나씩, B 트랙은 Sum과 Cout이 서로 다르게 나오는 두 입력 조합을 보여 준다.

도전 문제 4.9 프로토타입에서 실제 회로로 [선택]

우리 프로토타입을 실제 부품으로 만든다고 상상하자.

- (1) `decide` 함수를 제2장의 방법으로 논리회로로 그리시오.
- (2) 필요한 게이트의 개수와 종류를 세시오.
- (3) 같은 기능을 NAND 게이트만으로 만들면 게이트가 몇 개 필요한가?
- (4) 프로토타입에서는 체크박스가 센서 역할을 했다. 실제로 만든다면 각 입력에 어떤 센서를 쓸 수 있을지, 그 센서가 정말 “예/아니오”로 답할 수 있는지 검토하시오.
- (5) 실제 센서는 종종 틀린 값을 보낸다. 센서 하나가 고장 났을 때 우리 장치는 어떻게 동작하는가? 제1장의 XOR를 이용한 센서 불일치 감지를 붙일 수 있겠는가?

A. 수학 활동 정답 및 해설

A.1. 예제 1.1 스마트 복도 조명의 세 가지 설계

입력		출력 L		
P	D	설계 1 ($L = P$)	설계 2 ($L = D$)	설계 3 ($L = P \wedge D$)
0	0	0	0	0
0	1	0	1	0
1	0	1	0	0
1	1	1	1	1

- (1) 설계 2. $(P, D) = (0, 1)$ 인 행에서 사람이 없는데도 $L = 1$ 이다. 밤새 아무도 없는 복도에 조명이 켜져 있다.
- (2) 설계 1. $(P, D) = (1, 0)$ 인 행에서 밝은 낮인데도 $L = 1$ 이다.
- (3) 설계 3. $L = 1$ 인 행이 하나뿐이며, 그 행은 사람이 있고 어두운 경우다. 즉 조명이 실제로 필요한 상황에서만 켜진다.
- (4) 예를 들어 $L = P \wedge D \wedge M$ (M : 관리자가 허가했다)으로 바꾸면, 관리자가 허가하지 않은 시간에는 사람이 있고 어두워도 조명이 켜지지 않는다. 안전 문제가 생기고, 사용자는 결국 장치를 끄고 수동 스위치를 쓰게 된다. 조건을 AND로 더할 때마다 작동하는 경우의 수는 줄어들므로, “꼭 필요한 조건인가”를 매번 확인해야 한다.

A.2. 예제 1.2 우산 챙김 알림 장치의 진리표

R	O	U	$\neg U$	$A = R \wedge O \wedge \neg U$
0	0	0	1	0
0	0	1	0	0
0	1	0	1	0
0	1	1	0	0
1	0	0	1	0
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0

$A = 1$ 인 행은 $(R, O, U) = (1, 1, 0)$ 하나뿐이다. AND로 세 조건을 묶었으므로 세 조건이 동시에 만족되는 경우만 남는다.

검토 질문의 답. 네 질문 모두 “없다”이다. $R = 0$ 인 네 행과 $O = 0$ 인 네 행은 모두 $A = 0$ 이고, $U = 1$ 인 네 행도 모두 $A = 0$ 이다. 그리고 경고가 필요한 상황, 즉 비가 오고 외출하며 우산이 없는 유일한 행에서 $A = 1$ 이다.

A.3. 예제 1.3 스마트 사물함 출입 제어

C	T	E	$C \wedge T$	$O = (C \wedge T) \vee E$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

- (2) 출력이 1인 행은 001, 011, 101, 110, 111의 다섯 개다. 각 행에서만 참이 되는 식을 만들어 $\vee$ 로 연결하면 다음과 같다.

$$O \equiv (\neg C \wedge \neg T \wedge E) \vee (\neg C \wedge T \wedge E) \vee (C \wedge \neg T \wedge E) \vee (C \wedge T \wedge \neg E) \vee (C \wedge T \wedge E)$$

- (3) 위 식을 정리하면 원래 식과 같아진다. 앞의 세 항은 C 와 T 의 네 조합 중 세 가지에 E 가 붙은 것이고, 마지막 두 항은 $C \wedge T$ 에 $\neg E$ 와 E 가 붙은 것이다. 따라서

$$O \equiv E \vee (C \wedge T)$$

가 되어 $(C \wedge T) \vee E$ 와 동치다. 모든 행에서 두 식의 값이 같음을 표에서 직접 확인할 수도 있다.

- (4) **사용자 관점:** 교사가 자리에 없거나 승인 장치가 고장 나면, 정말 급한 상황에서 사물함을 열 수 없다. 우회 조건이 오히려 병목이 된다.

보안 관점: $E = 1$ 이면 C 와 T 가 무엇이든 열린다. 승인 버튼이 노출되어 있거나 승인 신호를 훔쳐 낼 수 있다면, 카드 확인과 시간 제한이 아무 역할도 하지 못한다.

- (5) 정답은 여럿이다. 대표적인 개선 두 가지를 든다.

$$\text{개선 A: } O = (C \wedge T) \vee (C \wedge E) \quad \text{개선 B: } O = (C \wedge T) \vee (E_1 \wedge E_2)$$

개선 A는 긴급 상황에도 학생 카드 확인을 요구한다. “누가 열었는가”가 기록되므로 무단 개방이 어렵다. 다만 카드를 잃어버린 학생은 여전히 열 수 없다.

개선 B는 서로 다른 두 사람의 승인을 함께 요구한다. 한 사람의 실수나 도용만으로는 열리지 않는다. 대신 두 사람을 동시에 찾아야 하므로 시간이 더 걸린다.

어느 쪽이 나은지는 사물함에 무엇이 들어 있는지에 따라 달라진다. 개선안을 제안한 뒤에는 반드시 진리표를 다시 만들어, “정말 급할 때 열리는가”와 “권한 없이 열리지 않는가”를 모두 확인해야 한다.

A.4. 예제 1.4 같은 기능, 다른 설계

G	H	$G \vee H$	$\neg G$	$\neg H$	$\neg G \wedge \neg H$	$\neg(\neg G \wedge \neg H)$
0	0	0	1	1	1	0
0	1	1	1	0	0	1
1	0	1	0	1	0	1
1	1	1	0	0	0	1

세 번째 열과 마지막 열이 네 행 모두에서 같으므로 두 설계는 동치다. 따라서 $G \vee H \equiv \neg(\neg G \wedge \neg H)$ 이다.

부품 비교. 설계 A는 OR 게이트 1개(종류 1가지)를 쓴다. 설계 B는 NOT 게이트 3개와 AND 게이트 1개, 모두 4개(종류 2가지)를 쓴다. 기능은 같지만 부품 수는 네 배다. 그런데도 설계 B가 의미 있는 이유는, OR 게이트를 구할 수 없는 상황에서도 NOT과 AND만으로 같은 기능을 만들 수 있음을 보여 주기 때문이다.

A.5. 도전 문제 1.2 XOR를 NOT과 AND로

- (1) $P \oplus Q = 1$ 인 입력은 $(P, Q) = (0, 1)$ 과 $(1, 0)$ 이다.
- (2) $(P, Q) = (1, 0)$ 에서만 참인 식은 $P \wedge \neg Q$ 이고, $(P, Q) = (0, 1)$ 에서만 참인 식은 $\neg P \wedge Q$ 이다.
- (3) 두 경우 중 하나라도 참이면 $P \oplus Q$ 가 참이므로

$$P \oplus Q \equiv (P \wedge \neg Q) \vee (\neg P \wedge Q)$$

- (4) $A = P \wedge \neg Q$, $B = \neg P \wedge Q$ 로 두고 $A \vee B \equiv \neg(\neg A \wedge \neg B)$ 를 적용하면

$$P \oplus Q \equiv \neg(\neg(P \wedge \neg Q) \wedge \neg(\neg P \wedge Q))$$

이므로 $P \oplus Q$ 는 $\neg$ 와 $\wedge$ 만으로 표현된다.

A.6. 연습문제 1.3 진리표에서 논리식 구성

$f(P, Q, R) = 1$ 인 행은 $(P, Q, R) = (0, 0, 1), (1, 0, 1), (1, 1, 0), (1, 1, 1)$ 이다. 각 행에서만 참이 되는 식을 만들어 $\vee$ 로 연결하면 다음과 같다.

$$f \equiv (\neg P \wedge \neg Q \wedge R) \vee (P \wedge \neg Q \wedge R) \vee (P \wedge Q \wedge \neg R) \vee (P \wedge Q \wedge R)$$

이 식은 더 짧게 정리할 수 있다. 앞의 두 항은 $R = 1$, $Q = 0$ 일 때 P 의 값과 무관하게 참이므로 $\neg Q \wedge R$ 이고, 뒤의 두 항은 $P = 1$, $Q = 1$ 일 때 R 의 값과 무관하게 참이므로 $P \wedge Q$ 이다.

$$f \equiv (\neg Q \wedge R) \vee (P \wedge Q)$$

게이트로 만들면 NOT 1개, AND 2개, OR 1개로 충분하다. 정리하기 전 식은 NOT 4개, AND 8개, OR 3개가 필요하므로, 같은 기능을 훨씬 적은 부품으로 만든 것이다.

마지막으로 $\vee$ 를 없애면

$$f \equiv \neg(\neg(\neg Q \wedge R) \wedge \neg(P \wedge Q))$$

이므로 f 는 $\neg$ 와 $\wedge$ 만으로 표현된다. $\square$

A.7. 예제 2.1 스마트 과학실 안전관리 시스템

G	H	P	$G \vee H$	$A = (G \vee H) \wedge P$	$F = G \vee H$
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	1	0	1
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	1	1

- (2) $A = 1$ 인 행은 3개, $F = 1$ 인 행은 6개다. F 는 위험이 감지되면 언제나 작동하지만, A 는 위험이 감지되고 또한 사람이 있을 때만 작동하기 때문이다. $\wedge P$ 가 붙으면서 여섯 경우 중 사람이 있는 세 경우만 남았다.
- (3) $A = 0$ 이고 $F = 1$ 인 행은 $(G, H, P) = (0, 1, 0), (1, 0, 0), (1, 1, 0)$ 의 세 개다. 모두 “실험실에 위험이 있지만 사람은 없는” 상황이다. 이때 정보를 올려도 들을 사람이 없으므로 정보는 정지하고, 환풍기는 가스를 빼내기 위해 작동한다. 쉬는 시간이나 방과 후에 실험실에서 가스가 새는 상황이 여기에 해당한다.
- (4) 회로는 그림 2.3과 같다. OR 게이트의 출력을 팬아웃으로 갈라 환풍기와 AND 게이트에 함께 보내면 OR 게이트 하나로 두 출력을 모두 만들 수 있다. 필요한 게이트는 OR 1개와 AND 1개, 모두 2개다. 팬아웃을 쓰지 않고 OR 게이트를 두 개 놓으면 게이트가 3개가 된다.

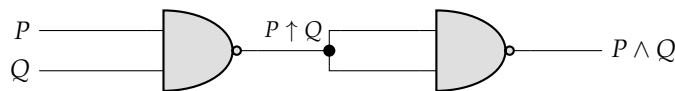
A.8. 예제 2.2 낭비를 찾아내기

D	W	$D \wedge W$	$L = (D \wedge W) \vee D$	D
0	0	0	0	0
0	1	0	0	0
1	0	0	1	1
1	1	1	1	1

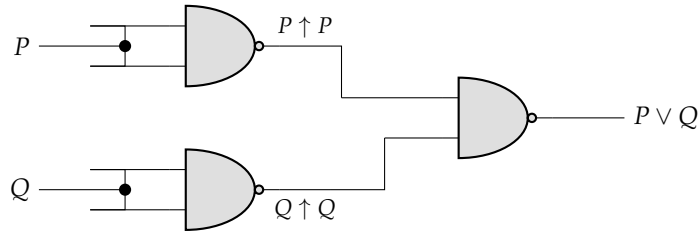
- (1) 네 번째 열과 다섯 번째 열이 모두 같으므로 $L \equiv D$ 이다. 이는 $P \vee (P \wedge Q) \equiv P$ 에서 $P = D, Q = W$ 인 경우다.
- (2) 보행자 센서 W 는 출력에 아무 영향을 주지 못한다. 센서를 달았지만 그 값이 무엇이든 결과가 같으므로, 실제로는 “어두우면 항상 켜다”는 장치다. 보행자가 없는 새벽에도 계속 켜져 있다.
- (3) AND 게이트 1개와 OR 게이트 1개를 모두 없앨 수 있고, 보행자 센서까지 뺄 수 있다. 게이트 2개와 센서 1개가 절약된다.
- (4) $L = D \wedge W$ 로 바꾸면 $(D, W) = (1, 0)$ 인 행에서 L 이 1에서 0으로 바뀐다. 즉 어둡지만 지나가는 사람이 없을 때 조명이 꺼진다. 네 경우 중 켜지는 경우가 두 개에서 한 개로 줄어드므로 전력 소비가 준다. 다만 사람을 감지하는데 시간이 걸리면 발밑이 잠깐 어두울 수 있으므로, 실제 제품에서는 감지 후 몇 초 동안 켜 두는 방식을 함께 쓴다.

A.9. 예제 2.3 NAND만으로 AND와 OR 만들기

- (1) $P \wedge Q \equiv (P \uparrow Q) \uparrow (P \uparrow Q)$. NAND 2개.



(2) $P \vee Q \equiv (P \uparrow P) \uparrow (Q \uparrow Q)$. NAND 3개.



(3) 기본 게이트로 만들면 AND는 1개, OR는 1개다. NAND만 쓰면 각각 2개와 3개이므로 게이트 개수는 늘어난다. 그러나 게이트 종류는 1가지로 줄어든다. NOT까지 포함하면 기본 게이트는 세 종류를 갖추어야 하지만, NAND 방식은 한 종류만 사서 쌓아 두면 된다.

A.10. 예제 2.4 합과 올림을 논리식으로

- (1) Sum 열은 두 입력이 서로 다를 때만 1이므로 **XOR**와 같다.
- (2) Carry 열은 두 입력이 모두 1일 때만 1이므로 **AND**와 같다.
- (3) $\text{Sum} = A \oplus B$, $\text{Carry} = A \wedge B$

학생이 “1 + 1은 2인데 왜 0인가”를 물으면, 이진수에서 2는 $10_{(2)}$ 이므로 그 자리에 남는 숫자가 0이고 윗자리로 1이 올라가는 것임을 십진수의 7 + 5와 나란히 놓고 설명한다. 십진수에서도 7 + 5의 그 자리 숫자는 2이고 올림이 1이다.

A.11. 연습문제 2.3 전가산기의 진리표

A	B	C_{in}	$A + B + C_{in}$	Sum	Cout
0	0	0	0	0	0
0	0	1	1	1	0
0	1	0	1	1	0
0	1	1	2	0	1
1	0	0	1	1	0
1	0	1	2	0	1
1	1	0	2	0	1
1	1	1	3	1	1

- (1) Sum이 1인 행은 세 입력 중 1의 개수가 홀수(한 개 또는 세 개)일 때다. 세 값을 XOR로 이은 $A \oplus B \oplus C_{in}$ 이 정확히 이 뜻이다.
- (2) Cout이 1인 행은 1의 개수가 두 개 이상일 때다. 세 개를 더해 2 이상이 되면 윗자리로 올림이 생기기 때문이다.

네 번째 열 $A + B + C_{in}$ 과 $\text{Sum} + 2\text{Cout}$ 을 비교하면 여덟 행 모두에서 값이 같다. 이 회로가 정말 덧셈을 한다는 확인이다.

A.12. 도전 문제 2.2 우리 팀 회로를 NAND만으로

정답은 팀마다 다르다. 다음 방법으로 진행하면 어떤 회로든 변환할 수 있다.

- 단계 1. 원래 회로의 NOT 게이트를 각각 NAND 1개와 팬아웃으로 바꾼다.
- 단계 2. AND 게이트를 각각 NAND 2개로 바꾼다.

단계 3. OR 게이트를 각각 NAND 3개로 바꾼다.

단계 4. 연속된 NOT 두 개가 생기면 지운다. $\neg\neg P \equiv P$ 이므로 NAND 2개를 절약할 수 있다.

예를 들어 우산 알림 회로 $A = R \wedge O \wedge \neg U$ 는 NOT 1개와 AND 2개를 쓰므로, 단순 변환하면 $\text{NAND } 1 + 2 + 2 = 5$ 개가 된다. 그런데 $\neg U$ 를 만드는 NAND의 출력을 그다음 NAND에 바로 넣으면 NOT과 AND가 이어지는 자리에서 두 번의 부정이 상쇄되어 NAND 4개로 줄일 수 있다. 게이트 수를 줄이는 최적화와 종류를 줄이는 표준화 가운데 우리 발명품에 어느 쪽이 중요한지 팀에서 판단하여 쓰면 된다.

B. Python Lab 정답 및 예상 출력

B.1. 실습 문제 3.2 세 가지 연산

01_boolean_practice.py의 예상 출력은 본문 실습 문제 3.2에 실린 것과 같다. True and False는 False, True or False는 True, not True는 False이다. 두 입력 진리표의 세 열은 제1장의 NOT, AND, OR 진리표와 정확히 일치한다.

B.2. 실습 문제 3.3 값을 바꾸어 실험하기

person_here와 is_dark의 네 조합에 대한 결과는 다음과 같다.

person_here	is_dark	hallway_light	출력 문구
False	False	False	조명을 켜지 않는다
False	True	False	조명을 켜지 않는다
True	False	False	조명을 켜지 않는다
True	True	True	조명을 켜다

확인 질문. and를 or로 바꾸면 두 번째와 세 번째 행이 True로 바뀐다. 켜지는 경우가 하나에서 세 개로 늘어나므로 전기를 더 켜. 사람이 없는 밤에도, 사람이 있는 밝은 낮에도 켜진다.

B.3. 실습 문제 3.4 진리표 자동 생성과 테스트

decide 함수가 옳게 작성되면 8/8이 통과하고, 출력된 진리표는 부록 A의 예제 1.2 진리표와 한 행씩 일치한다. TEST_CASES의 정답은 다음과 같다.

```
1 # 제품 시험표: 종이 진리표에서 예상 출력을 옮겨 적은 것
2 # ((비, 외출, 우산), 예상 출력)
3 # =====
4
5 TEST_CASES = [
6     ((False, False, False), False),
7     ((False, False, True), False),
8     ((False, True, False), False),
9     ((False, True, True), False),
10    ((True, False, False), False),
11    ((True, False, True), False),
12    ((True, True, False), True),
13    ((True, True, True), False),
14 ]
```

B.4. 실습 문제 3.6 오작동 수리 절차

두 고장 난 함수의 테스트 결과와 실패 원인은 다음과 같다. code/solutions/truth_table_solution.py를 실행하면 실패한 행마다 이유가 함께 출력된다.

```
[3] 오작동 예 1 (and 대신 or) - 6개 행에서 실패
테스트 결과: 2 / 8 통과
실패 - 입력 (0 0 0) : 예상 0, 실제 1 <- 비도 오지 않고 외출도 하지 않는데 경고한다.
실패 - 입력 (0 1 0) : 예상 0, 실제 1 <- 비가 오지 않는데 경고한다.
실패 - 입력 (0 1 1) : 예상 0, 실제 1 <- 비가 오지 않는데 경고한다.
실패 - 입력 (1 0 0) : 예상 0, 실제 1 <- 외출하지 않는데 경고한다.
실패 - 입력 (1 0 1) : 예상 0, 실제 1 <- 외출하지 않는데 경고한다.
```

실패 - 입력 (1 1 1) : 예상 0, 실제 1 <- 이미 우산을 가지고 있는데 경고한다.
 진단: or 는 조건 중 하나만 참이어도 작동하므로
 불필요한 경고가 매우 많아진다. and 로 고친다.

[4] 오작동 예 2 (not 빠뜨림) - 2개 행에서 실패
 테스트 결과: 6 / 8 통과
 실패 - 입력 (1 1 0) : 예상 1, 실제 0 <- 정말 필요한 상황인데 경고하지 않는다.
 실패 - 입력 (1 1 1) : 예상 0, 실제 1 <- 이미 우산을 가지고 있는데 경고한다.
 진단: 우산을 이미 가진 경우에 경고하고,
 정말 필요한 경우에는 침묵한다. not U 로 고친다.

고장 난 함수	통과	무엇을 어떻게 고치는가
decide_broken_1	2/8	or를 and로 바꾼다. or는 조건 하나만 참이어도 작동하므로 쓸데없는 경고가 여섯 경우에서 발생한다.
decide_broken_2	6/8	has_umbrella를 not has_umbrella로 바꾼다. 조건의 뜻이 반대로 되어, 우산이 있을 때 경고하고 정작 없을 때 침묵한다.

어느 고장이 더 위험한가. decide_broken_1은 실패 개수가 많아 눈에 금방 띈다. 반면 decide_broken_2는 6/8이 통과하므로 대충 보면 잘 작동하는 것처럼 보이지만, 정작 필요한 상황에서 침묵한다. 안전 장치라면 후자가 훨씬 위험하다. 그래서 통과율이 아니라 어느 행이 실패했는지를 보아야 한다.

B.5. 도전 문제 3.7 진리표에서 논리식 되찾기

TEST_CASES에서 예상 출력이 True인 행을 찾고, 그 행에서만 참이 되는 식을 and와 not으로 만들고, or로 연결하면 된다. 우산 알림 장치는 True인 행이 하나뿐이므로 결과가 원래 식과 같다.

```
1 def decide(rain, going_out, has_umbrella) -> bool:
2     return (rain and going_out and not has_umbrella)
```

연습문제 1.3의 진리표처럼 True인 행이 네 개인 경우에는 or가 필요하다.

```
1 def f(p, q, r) -> bool:
2     return ((not p and not q and r)
3             or (p and not q and r)
4             or (p and q and not r)
5             or (p and q and r))
```

부록 A에서 정리한 짧은 식으로 써도 결과는 같다. 두 함수가 여덟 행 모두에서 같은 값을 내는지 테스트 코드로 확인해 보면 논리식 정리가 옳았음을 프로그램으로 검증한 셈이 된다.

```
1 def f_short(p, q, r) -> bool:
2     return (not q and r) or (p and q)
```

B.6. Lab 정답 파일

파일	내용
code/solutions/truth_table_solution.py	우산 알림 진리표 정답, 완성된 TEST_CASES, 두 오작동 함수의 실패 행과 이유
code/solutions/umbrella_solution.py	04_invention_starter_1out.py의 학생 수정 영역을 우산 알림 장치로 채운 정답. 실행하면 스스로 검증한다
code/solutions/safety_system_solution.py	05_invention_starter_2out.py의 학생 수정 영역을 과학실 안전 시스템으로 채운 정답
code/solutions/full_adder_solution.py	(B 트랙) 전가산기 정답, 혼한 오작동 두 가지, 선택 도전인 두 자리 덧셈기

우산 알림 장치의 학생 수정 영역 정답은 다음과 같다.

```
1  TITLE = "우산 챙김 알림 장치"
2  DESCRIPTION = "센서를 켜고 끄면 출력이 즉시 갱신됩니다."
3
4  INPUT_LABELS = [
5      "비가 온다 (R)",
6      "외출할 예정이다 (O)",
7      "우산을 가지고 있다 (U)",
8  ]
9
10 OUTPUT_LABEL = "우산 알림"
11 MESSAGE_ON = "우산을 챙기세요!"
12 MESSAGE_OFF = "현재 알림이 필요하지 않습니다."
13
14
15 def decide(rain: bool, going_out: bool, has_umbrella: bool) -> bool:
16     """판단 규칙: A = R and O and (not U)"""
17     return rain and going_out and not has_umbrella
18
19
20 TEST_CASES = [
21     ((False, False, False), False),
22     ((False, False, True), False),
23     ((False, True, False), False),
24     ((False, True, True), False),
25     ((True, False, False), False),
26     ((True, False, True), False),
27     ((True, True, False), True),
28     ((True, True, True), False),
29 ]
```

과학실 안전 시스템의 판단 함수와 시험표는 다음과 같다.

```
1  def decide(gas: bool, high_temp: bool, person: bool):
2      """A = (G or H) and P, F = G or H"""
3      danger = gas or high_temp
4      alarm = danger and person
5      fan = danger
6      return alarm, fan
7
8
9  TEST_CASES = [
10     ((False, False, False), (False, False)),
```

```

11 ((False, False, True), (False, False)),
12 ((False, True, False), (False, True)),
13 ((False, True, True), (True, True)),
14 ((True, False, False), (False, True)),
15 ((True, False, True), (True, True)),
16 ((True, True, False), (False, True)),
17 ((True, True, True), (True, True)),
18 ]

```

B.7. B 트랙 전가산기의 혼한 오작동

full_adder_solution.py에는 학생들이 실제로 자주 만드는 오작동 두 가지가 함께 들어 있다. 두 검사 결과를 비교해 보면 원인을 빠르게 짚을 수 있다.

오작동	통과	원인과 수정
올림을 OR 대신 AND로 모음	4/8	두 반가산기에서 동시에 올림이 나는 일은 없으므로 AND로 모으면 Cout이 항상 0이 된다. Cout이 원래 0인 네 행 000, 001, 010, 100만 통과한다. $(a \text{ and } b) \text{ or } (s1 \text{ and } \text{carry_in})$ 으로 고친다.
두 번째 반가산기에서 C_{in} 누락	4/8	$C_{in} = 1$ 인 네 행에서 Sum이 틀린다. $\text{digit} = \text{xor}(s1, \text{carry_in})$ 으로 고친다.

두 오작동 모두 덧셈 검사에서도 함께 실패한다. 반대로 논리식은 맞는데 TEST_CASES를 잘못 옮겨 적은 경우에는 진리표 비교만 실패하고 덧셈 검사는 통과한다. 어느 검사가 실패했는지를 먼저 물어보면 학생이 스스로 원인을 좁힐 수 있다.

B.8. 도전 문제 4.4 두 자리 이진 덧셈기

핵심은 오른쪽 자리의 Cout을 왼쪽 자리의 C_{in} 으로 넘기는 것이다.

```

1 def add_two_bits(a1: bool, a0: bool, b1: bool, b0: bool):
2     """두 자리 이진수 (a1 a0) 와 (b1 b0) 를 더한다."""
3     digit0, carry0 = decide(a0, b0, False)
4     digit1, carry1 = decide(a1, b1, carry0)
5     return carry1, digit1, digit0

```

- (1) 가장 오른쪽 자리에는 넘어올 아랫자리가 없으므로 $C_{in} = 0$, 즉 False를 넣는다.
- (2) 두 자리 이진수의 최대값은 $11_{(2)} = 3$ 이므로 합은 최대 $3 + 3 = 6$ 이고, $6 = 110_{(2)}$ 이므로 세 자리가 필요하다. 그래서 왼쪽 자리의 Cout이 세 번째 자리가 된다.
- (3) 16가지 조합 모두에서 $(A_1A_0)_{(2)} + (B_1B_0)_{(2)} = (\text{carry1 digit1 digit0})_{(2)}$ 가 성립한다. 정답 파일이 이 표를 출력한다.
- (4) 자리를 늘리려면 decide 호출을 한 번 더 추가하고, 바로 아랫자리의 Cout을 그 호출의 C_{in} 으로 넘기면 된다. 이렇게 올림을 차례로 흘러보내는 방식을 리플 캐리(ripple carry)라고 한다.

C. tkinter 교사용 운영 안내

C.1. 수업 전 준비

- (1) 실습용 컴퓨터에서 `python --version`과 `python -m tkinter`가 동작하는지 미리 한 대 이상에서 확인한다. `python`이 없으면 `py`를 사용하도록 안내한다.
- (2) `code` 폴더를 공유 폴더나 USB로 배포한다. 학생이 쓰기 권한이 없는 위치에서 실행하면 저장이 되지 않으므로, 각자 자기 폴더에 복사하도록 한다.
- (3) 1일차에는 모든 학생이 00, 01, 02, 03 공통 파일을 각자 복사하고 실행하게 한다. 빠른 학생은 결과를 설명할 수 있지만 다른 학생의 코드를 대신 작성하지 않게 한다.
- (4) 2일차에는 아이디어 공유와 팀 구성이 끝나고 팀 아이디어가 확정된 뒤에만 팀별로 해당 스타터 파일을 복사하고 이름을 바꾸게 한다. 원본을 직접 고치면 다른 팀이 쓸 수 없다.
- (5) `06_safety_system_example.py`를 미리 실행해 두면 2차시 도입에서 완성 예시를 바로 보여 줄 수 있다.

운영 원칙. 2일차의 개인 아이디어 발표는 평가가 아니라 팀을 만들기 위한 정보 공유다. 모든 발표 뒤 약 5분 동안 아이디어를 둘러보게 하고, 그다음 2 ~ 4명씩 팀을 만든다. 특정 아이디어에 학생이 지나치게 몰리면 비슷한 주제의 팀을 둘 이상 만들거나 관점을 나누어 팀 크기를 조정한다.

C.2. 자주 발생하는 오류와 대응

증상	원인	대응
python 명령을 찾을 수 없다	명령 이름이 다르거나 PATH에 없다	py로 실행한다
ModuleNotFoundError: No module named 'tkinter'	tkinter가 빠진 배포판을 설치했다	python.org 배포판으로 다시 설치한다
창이 열리지 않는다	다른 창 뒤에 가려졌거나, 오류로 즉시 종료되었다	작업 표시줄을 확인하고, 터미널의 오류 메시지를 읽는다
IndentationError	들여쓰기가 어긋났다	return 앞을 공백 4 칸으로 맞춘다. 탭과 공백을 섞지 않는다
SyntaxError: invalid syntax	따옴표, 괄호, 콜론이 빠졌다	오류 줄 번호의 앞뒤 줄을 함께 확인한다
TypeError: decide() takes 3 positional arguments but 4 were given	센서 개수와 매개변수 개수가 다르다	INPUT_LABELS, decide(), TEST_CASES의 개수를 맞춘다
테스트 안내 메시지가 형식 오류를 알린다	TEST_CASES의 괄호나 심표가 빠졌다	메시지에 나온 번호의 줄을 고친다. 한 줄은 ((입력, 입력, 입력), 예상출력), 형태다
고쳤는데 결과가 그대로다	저장하지 않았거나 이전 창이 아직 열려 있다	저장하고, 열려 있는 창을 닫은 뒤 다시 실행한다
한글이 깨져 보인다	파일이 UTF-8이 아닌 인코딩으로 저장되었다	편집기에서 UTF-8로 다시 저장한다

C.3. 진행 속도가 다른 모듈 지원

느린 모듈.

- (1) 입력을 두 개로 줄이고 TEST_CASES를 네 행으로 만들어 먼저 작동하는 프로그램을 완성하게 한다. 작동하는 것을 한 번 본 뒤에 확장하는 편이 훨씬 빠르다.
- (2) 논리식이 막히면 진리표에서 출력이 1인 행 하나만 골라 그 행에 해당하는 조건을 and로 이어 보게 한다.
- (3) 팀의 최종 진리표를 확정하고 TEST_CASES로 옮긴 뒤, decide를 구현하고 마지막에 GUI를 고치게 한다.
- (4) 오류 메시지를 대신 읽어 주지 말고, 오류의 마지막 줄과 줄 번호를 함께 보게 한다.

B 트랙 모듈.

- (1) 주제 선정은 필요 없지만 곧바로 구현에 들어가지는 않는다. “이 장치를 누가 어떻게 쓰는가”, 입력과 출력, 최종 진리표를 팀이 확인하고 TEST_CASES를 먼저 작성하게 한다. 계산 장치도 사용자가 있는 물건이다.
- (2) xor 함수를 지우고 직접 다시 쓰게 하는 것이 이 트랙의 핵심 활동이다. ^ 연산자를 찾아 쓰는 학생에게는 1일차에 유도한 식을 코드로 옮기는 것이 과제임을 다시 알린다.
- (3) 두 검사 중 하나만 실패했다면 원인이 좁혀진다. 진리표 비교만 실패하면 TEST_CASES를 잘못 옮긴 것이고, 둘 다 실패하면 논리식이 틀린 것이다.
- (4) 일찍 끝난 모둠에게는 도전 문제 4.4의 두 자리 덧셈기를 준다.

빠른 모둠 (A 트랙).

- (1) 05_invention_starter_2out.py로 출력을 두 개로 늘리고, 두 출력의 조건식을 다르게 설계하게 한다.
- (2) 정상, 주의, 위험 세 단계를 구분하는 상태 표시를 추가하게 한다. safety_level 함수가 참고가 된다.
- (3) 도전 문제 4.9의 회로 변환과 센서 검토를 진행하게 한다.
- (4) 수학 심화를 원하면 부록 D의 NAND 전용 회로를 준다. 전가산기에 관심을 보이면 B 트랙 파일을 열어 보게 해도 좋다.
- (5) 다른 모둠의 프로토타입을 교차 테스트하고 개선안을 문서로 정리하게 한다.

C.4. 강의안의 코드와 실습 파일 동기화

이 강의안에 실린 모든 코드와 콘솔 출력은 code 폴더의 실제 파일에서 자동으로 뽑아낸 것이다. code의 .py 파일을 수정했다면 다음 명령을 한 번 실행한 뒤 LaTeX를 다시 컴파일한다.

```
python tools/make_excerpts.py
```

이 명령은 지정된 구간을 찾아 tools/excerpts 폴더에 발췌 파일을 만들고, 콘솔 출력이 필요한 곳은 실제로 프로그램을 실행해 결과를 받아 온다. 구간을 찾지 못하면 어떤 발췌가 실패했는지 알려 주고 멈추므로, 강의안의 코드와 실습 파일이 서로 달라지는 일이 없다.

문서 컴파일은 다음과 같이 한다. -shell-escape는 필요하지 않다.

```
latexmk Main.tex
```

D. 수학 심화

이 부록은 수학적 확장을 원하는 학생을 위한 것이다. tkinter 프로토타입의 필수 조건이 아니다.

D.1. XOR는 범용 게이트인가

심화 문제 D.1

XOR 게이트만으로 모든 불 함수를 만들 수 있는가? 만들 수 있음을 증명하거나, 만들 수 없는 기본 게이트를 제시하시오.

답. XOR는 범용 게이트가 아니다.

[증명] XOR만을 여러 번 조합하면 항상

$$x_{i_1} \oplus x_{i_2} \oplus \cdots \oplus x_{i_k}$$

형태의 식만 얻어진다. 즉 변수들의 XOR 조합, 다시 말해 선형 불 함수만 표현할 수 있다.

범용 게이트가 되려면 $\wedge$ 를 표현할 수 있어야 한다. 그런데 $A \oplus A = 0$ 이므로 같은 변수를 두 번 XOR하면 항상 0이 된다. 따라서 XOR만으로는 상수 1을 만들 수 없고, 상수 1이 없으면 $\neg A = A \oplus 1$ 도 만들 수 없다.

반면 AND는 $A \wedge A = A$ 를 만족하고, 특히 $A = 1$ 이면 $1 \wedge 1 = 1$ 이다. 모든 입력이 0일 때 XOR 조합의 값은 항상 0이지만, $\neg A$ 는 1이 되어야 한다. 그러므로 $\neg$ 와 $\wedge$ 를 XOR 조합으로 만들 수 없고, XOR는 범용 게이트가 아니다. $\square$

참고로 $\text{NOR}(\neg(P \vee Q))$ 는 NAND와 마찬가지로 범용 게이트다.

D.2. 함수적 완전성과 범용 게이트

지금까지 확인한 내용을 정리하면 다음과 같다.

연산자 집합	함수적 완전성	근거
$\{\neg, \wedge, \vee\}$	완전	진리표의 각 행에서 최소항을 만들어 $\vee$ 로 연결하면 된다
$\{\neg, \wedge\}$	완전	드모르간 법칙으로 $\vee$ 를 없앨 수 있다
$\{\neg, \vee\}$	완전	드모르간 법칙으로 $\wedge$ 를 없앨 수 있다
$\{\uparrow\}$ (NAND)	완전	$\neg$ 와 $\wedge$ 를 NAND만으로 만들 수 있다
$\{\oplus\}$ (XOR)	완전하지 않음	선형 함수만 표현할 수 있고 상수 1을 만들 수 없다
$\{\wedge, \vee\}$	완전하지 않음	모든 입력이 0이면 출력이 항상 0이므로 $\neg$ 를 만들 수 없다

심화 문제 D.2

$\{\wedge, \vee\}$ 가 함수적으로 완전하지 않음을 위 표의 근거를 이용하여 증명하시오.

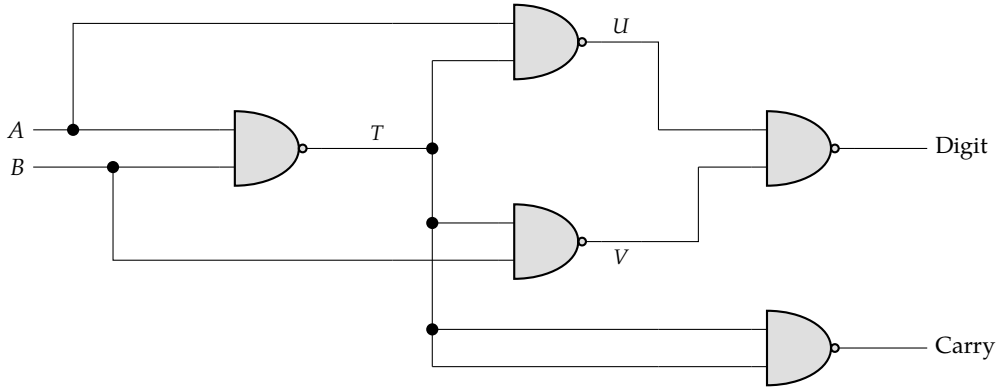
답. $\wedge$ 와 $\vee$ 만으로 만든 식은 모든 입력이 0일 때 반드시 0이 된다. 실제로 입력이 모두 0이면 $0 \wedge 0 = 0$, $0 \vee 0 = 0$ 이므로, 식을 어떻게 조합해도 중간 결과가 계속 0이다. 그런데 $\neg P$ 는 $P = 0$ 일 때 1이어야 한다. 따라서 $\neg$ 를 $\{\wedge, \vee\}$ 로 만들 수 없다. $\square$

D.3. NAND만으로 만든 반가산기

심화 문제 D.3

반가산기의 Digit과 Carry를 NAND 게이트만으로 구성하시오.

답. $\text{Digit} = A \oplus B$ 와 $\text{Carry} = A \wedge B$ 를 NAND만으로 만들면 NAND 5개로 충분하다. 중간 신호 $A \uparrow B$ 를 팬아웃으로 세 곳에 공유하는 것이 핵심이다.



첫 번째 NAND와 위쪽의 세 NAND, 총 4개의 NAND가 XOR를 만들어 Digit을 출력한다. 오른쪽 아래 NAND는 첫 번째 NAND의 출력 $T = A \uparrow B$ 를 두 입력에 함께 받아 다시 부정하므로 $\neg T = \neg(A \uparrow B) = A \wedge B$, 즉 Carry를 만든다.

D.4. 전가산기 설계

심화 문제 D.4

입력이 A, B, C_{in} 이고 출력이 Digit과 Carry인 전가산기(full adder)의 회로를 설계하시오.

답. 전가산기는 세 입력을 더하여 한 자리의 합 Digit과 올림 Carry를 출력한다.
먼저 A 와 B 를 더하는 반가산기를 생각하면

$$S_1 = A \oplus B, \quad C_1 = A \wedge B$$

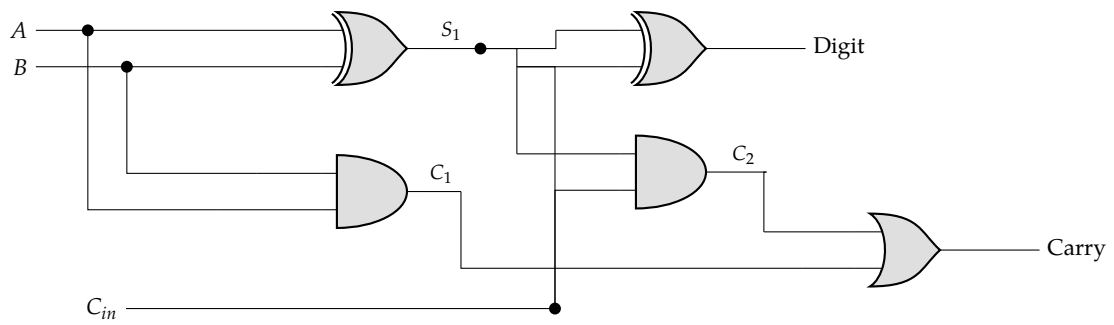
이다. 이제 S_1 에 이전 자리에서 올라온 올림 C_{in} 을 다시 더하면 최종 자릿수는

$$\text{Digit} = S_1 \oplus C_{in} = (A \oplus B) \oplus C_{in}$$

이고, 두 번째 덧셈에서 새로 생기는 올림은 $C_2 = S_1 \wedge C_{in}$ 이다. 최종 올림은 두 올림 중 하나라도 발생하면 되므로

$$\text{Carry} = C_1 \vee C_2 = (A \wedge B) \vee ((A \oplus B) \wedge C_{in})$$

이다. 회로로 나타내면 다음과 같다.



따라서 전가산기는 XOR 2개, AND 2개, OR 1개로 구성할 수 있다. 모든 게이트를 NAND로 바꿀 수 있으므로, NAND와 팬아웃만으로 여러 자리 이진수의 덧셈 회로를 만들 수 있고, 같은 원리로 컴퓨터 전체를 구성할 수 있다(Petzold, 2000). □

E. 평가 기준

프로토타입의 외형보다 논리적 정확성, 테스트, 사용자 관점의 개선을 더 높게 평가한다.

평가 항목	배점	평가 내용
설계 목표와 사용 맥락	15	(A) 해결하려는 생활 속 문제가 구체적이고 사용자가 분명한가. (B) 무엇을 계산하는 장치인지, 누가 어떻게 쓰는지 분명한가.
입력과 출력 조건의 타당성	15	입력이 “예/아니오”(또는 비트 0/1)로 답할 수 있는 조건인가. 출력이 장치가 실제로 내놓을 수 있는 값인가. 입력과 출력의 개수가 목표에 알맞은가.
진리표와 논리식의 정확성	20	모든 입력 조합이 빠짐없이 적혀 있는가. 자연어 조건과 논리식이 정확히 같은 뜻인가. 출력이 두 개인 경우 목적에 따라 조건식을 구분했는가.
Python 판단 함수 구현	15	<code>decide()</code> 가 논리식과 일치하는가. <code>and</code> , <code>or</code> , <code>not</code> 을 올바르게 사용했는가. 입력 개수가 세 곳에서 일치하는가.
전체 입력 테스트와 오작동 수정	15	<code>TEST_CASES</code> 를 진리표에서 옮겨 완성했는가. 전체 조합을 시험했는가. 실패한 행의 원인을 설명하고 고쳤는가. (B) 덧셈 검사까지 함께 통과시켰는가.
사용자 인터페이스의 명확성	10	입력 이름과 상태 표시를 설명 없이 이해할 수 있는가. 두 상태가 구분되는가. (A) 경고 문구가 사용자의 행동을 안내하는가. (B) 결과를 이진수와 십진수로 함께 읽을 수 있는가.
설계 개선과 발표	10	테스트나 교차 테스트 결과를 근거로 설계를 고쳤는가. 개선의 이유를 게이트 수, 전력, 사용자 관점으로 설명했다. 시연에서 서로 다른 결과가 나오는 입력 조합을 함께 보였는가.
합계	100	

E.1. 채점 시 유의점

- 색과 글꼴만 바꾼 것은 사용자 인터페이스 개선으로 인정하지 않는다. 사용자의 판단이나 행동이 달라져야 개선이다.
- 오작동을 발견하고 기록한 모듬은, 발견하지 못한 모듬보다 높게 평가한다. 오작동을 찾는 일이 곧 설계 능력이다.
- “정답 하나”를 요구하지 않는 항목이 있다. 스마트 사물함의 우회 조건, 환기 알림의 수업 중 작동 여부처럼 선택이 갈리는 문제는 선택의 근거를 평가한다.
- NAND 전용 회로와 두 자리 덧셈기는 가산점 요소이며, 필수 항목이 아니다.
- A 트랙과 B 트랙은 같은 배점으로 평가한다. B 트랙은 만들 장치가 정해져 있어 주제 선정 부담이 없는 대신, 두 가지 검증을 모두 통과시켜야 하므로 난이도가 낮지 않다. 두 트랙 모두 “논리식이 맞는가”와 “사용자가 이해할 수 있는가”를 같은 무게로 본다.

References

- Petzold, C. (2000). *Code: The Hidden Language of Computer Hardware and Software*. Microsoft Press, Redmond, WA.
- Python Software Foundation (2024a). The python language reference: Boolean operations. <https://docs.python.org/3/reference/expressions.html>.
- Python Software Foundation (2024b). tkinter: Python interface to tcl/tk. <https://docs.python.org/3/library/tkinter.html>.